

Where the Seconds Go:

A Speed Limit and a Measured Accounting for Mixture-of-Experts Decode on One Consumer GPU

Harshith Kantamneni
Independent Researcher

Abstract

A Mixture-of-Experts model too large for one consumer GPU keeps most experts in host memory and reads them as each token needs them. How fast could it decode, and what is the rest of the time made of? Belady’s optimum with bypass on the model’s own routing, at probed and datasheet rates, gives a speed limit in seconds: our llama.cpp expert cache reaches 25–43% of it on two RTX 5090 hosts, and 52 published measurements a median 14.5%. A Shapley accounting names routing foresight the largest missing piece where host memory binds; oracles in the engine measure it. Spent as the optimum does, admitting what it admits and reading each admitted expert once, foresight is worth 25–38% serialised and 42–67% overlapped; spent by Belady prefetch or by copying what the CPU has just served, it reads $1.7\text{--}2.5\times$ the optimum’s bytes and looked worthless where host memory is saturated. Nine models’ traces give the horizon a forecaster needs, $W_{50} \approx 0.59 (C/k)^{1.33}$ tokens, which speculative batching does not supply.

1 Introduction

Open-weight Mixture-of-Experts models now carry 30–120B parameters and activate a few billion per token; a desktop GPU has 24–32GB. Running such a model at batch 1 therefore means keeping most experts in host DRAM, and each decoded token then reads its routed experts from one of three places: GPU memory, host memory by the CPU that runs the expert, or host memory across PCIe into a GPU slot [17, 13]. A line of systems has built on this: GPU expert caches with demand fetch [8, 43, 35], CPU execution of misses [19, 5, 53, 15, 52, 55], permiss choices between the two [46], and expert prediction [16, 54, 41]. Each reports a speed-up over a baseline of its choosing. None states how fast its machine *could* decode the model within the same GPU memory, or how much of the remaining time any design could remove, and the trace-level evaluation contracts proposed for this field [51] count cache misses, not seconds.

This paper asks those two questions of a consumer machine and answers them in seconds. The answer to the first is a ceiling: the fewest host bytes any exact-routing policy can read, from Belady’s optimum with bypass on

the model’s own routing, over the probed host rate and the GPU’s datasheet rate. The answer to the second is an accounting of the gap to that ceiling, first as a model and then as a measurement: we built oracles of future routing into the engine and measured what foresight is worth under each way of spending it. Spent as the optimum spends it, admitting only what MIN with bypass admits and reading each admitted expert from host memory once, foresight pays at every budget, and most of what it is worth where host memory binds is the bytes it saves. Spent the way caches usually spend it, by a Belady prefetch that admits experts the optimum would bypass, or by the serve-then-copy admission of caches with CPU-executed misses, which reads an admitted expert twice, it costs $1.7\text{--}2.5\times$ the optimum’s bytes and looked worthless at the lowest budgets, in our own first attempts included. We work on rented RTX 5090, 4090 and 3090 hosts with desktop CPUs, with gpt-oss-120b [29] and Qwen3-30B-A3B [45], routing traces of nine models, and 52 published measurements from 13 sources, by the method of Hoefler and Belli [14]: a performance model with bounds, statistically sound comparisons, and every prediction committed to a public repository before the machine it concerns was started; 54% of 287 scored clauses held with their intervals (Appendix B). One example runs through the paper: gpt-oss-120b with 32 of each layer’s 128 experts resident, a quarter of its expert bytes, on an RTX 5090.

Contributions.

- **A speed limit in seconds** (Section 4), for any exact-routing system with C expert slots per layer, from the exact Belady optimum with bypass, two host read paths and measured ceilings, varied four ways and stated for pooled caches too; and the *host-memory law* that places an engine in its coordinates (Section 3): time per token as a GPU term plus host bytes over the probed rate of the paths that read them, predicted blind on 7 machines, re-scored with frozen constants on three more, and used by our cache to choose, per machine, which misses to copy and which to run on the CPU.
- **An accounting of the gap, predicted and then measured** (Section 5): a Shapley attribution over

five modelled fixes, and oracle policies in the engine that spend foresight four ways (fetched in the step, prefetched ahead of it, both on a paced copy path, and as a Belady prefetch), measured on one host at six cells beside the model’s values: at the four host-bound cells 34–49% of the gap is the bytes foresight saves, 8–23% the overlap it allows, and the 39–43% that remains is named.

- **What would recover it** (Section 6): across nine models’ routing traces, which online policy earns its reads, how fast the hot set drifts, how many tokens of foresight close half of the gap between the deployed policy and the optimum ($W_{50} \approx 0.59 (C/k)^{1.33}$), and why batched speculative verification cannot supply them.

The instrument is our llama.cpp expert cache (Section 2): per-layer slots, rationed admission, CPU helps the GPU signals, and demand copies split by the law; it runs 2.0–4.0 $\times$ stock llama.cpp and leads FreeToken [46] at 11 of 12 cells on two RTX 5090 hosts. Running missed experts on the CPU [19, 5], caching hot experts with CPU-executed misses [53, 15, 22], splitting a layer’s misses between CPU and PCIe from a bandwidth model [46, 55], a PCIe turnover ceiling [49], bytes over bandwidth as a bound [48], Belady with bypass as a cache-level oracle [51] and a table of systems against the all-in-VRAM speed [41] are all published; our system uses the first three, and what is new is the limit in seconds, the accounting and its measurement, the trace results, and the measurements.

2 Setting and Instrument

Workload. Single-request decode. gpt-oss-120b (MXFP4 experts; 36 MoE layers of 128 experts, top-4, 13.25 MB per expert) and Qwen3-30B-A3B in BF16 (48 layers of 128 experts, top-8, 9.44 MB per expert). Prompts are the 30 AIME-25 problems in FreeToken’s own benchmark format; each system decodes 256 tokens per problem, greedily, after one warm-up request on a fixed held-out text, in one session in which the cache carries over between problems. One client measures every system with FreeToken’s own metric, (tokens–1) over the time between the first and last token’s arrival. A *cell* is a model at a budget: the fraction of each layer’s experts resident on the GPU (11, 25 and 40% for gpt-oss; 12.5, 25 and 43.75% for Qwen3).

Machines and probes. RTX 5090 (32 GB, 1.79 TB/s datasheet) hosts with desktop CPUs, rented by the hour, each named here by a letter. Table 1 is measured on *host B*, a Ryzen 9 9950X3D rental whose card reports a 17001 MHz memory clock, 1.21 $\times$ the common clock (14001 MHz on 25 of the 26 other RTX 5090 jobs that recorded one) and whose probe read device memory 3% above the datasheet rate the limit assumes, and on *host*

S, a Ryzen 9 9950X rental at the common clock with a 22% slower CPU. The ablation and the long-output runs use *host A*, a second rental of the 9950X3D model at the common clock; the oracle measurements of Section 5 use three further rentals, *O1* (Ryzen 9 9950X), *O2* (Ryzen 9 7950X) and *O3* (Ryzen 9 9950X), whose host memory is weaker than host B’s. Each job first runs a probe (*concur*) that reads host memory from t CPU threads, over PCIe with zero-copy kernels and with the copy engine, and from both at once; FreeToken’s own probe (*ft bench bw*) runs too. We call the CPU rate at the helper thread count B_c , the zero-copy PCIe rate B_p and the combined rate B_{cp} , and the highest rate the probe reached by any method B_{host} . GPU clocks cannot be locked on these rentals; the cards’ power limits are in the released probe files (350–600 W).

The instrument. Our cache is a 3,500-line patch to llama.cpp. Each MoE layer has C GPU slots. Routing is decided on the GPU; resident experts run there, and each miss either runs on CPU helper threads, which the GPU signals through a mailbox in mapped memory, or is copied into a slot and run on the GPU (FETCH). Admission follows a decayed-frequency score (half-life 16 tokens, an admission margin $\kappa=1$; a llama.cpp pull request admits by a decayed heat with hysteresis in the same spirit [27]) and copies in the background, so an admitted miss is read from host memory twice, once by the CPU that serves it and once by its copy. Sampling runs on the GPU. The one per-machine decision, how many of a layer’s n missed experts to copy and how many to run on the CPU, is a table $f(n)$ computed from the probe by the law of Section 3 when the job starts. The cache serves single-token steps; prompts and batches above one run llama.cpp’s stock graph with every expert in host memory (79–83% of stock llama.cpp’s prefill speed, 77–79% of its decode at 2–4 sequences; Appendix F).

Equal memory, statistics and protocol. llama.cpp keeps whole layers’ experts on the GPU (`--n-cpu-moe n`), our cache keeps C experts of every layer, and FreeToken takes a cache rate; we match the number of experts on the GPU (within 2.5%) and report measured VRAM. Ratios are of mean speeds, paired by problem, with 95% percentile intervals from 10,000 bootstrap resamples; they cover problem sampling within one launch on one machine, not rentals (two rentals of one CPU model differed by 19–22% in decode speed and read 62 against 47 GB/s of host bandwidth). Where a system has variants (our FETCH tables, FreeToken’s offload and hybrid backends), a first launch picks each system’s faster variant per budget and a second launch, which the comparison uses, runs only those (one row of Table 1 has no second launch; on host S FreeToken’s variants were carried over from host B’s selection rather than re-picked). Every job’s predictions are in the header of its script, committed and pushed before the machine starts; Appendix B scores every clause

Table 1: Decode speed at equal GPU expert memory on two RTX 5090 hosts. 30 AIME-25 problems, first 256 decode tokens, greedy, session; our cache uses the FETCH split computed from each machine’s bandwidth probe; FreeToken uses its faster backend per budget, picked on host B on a separate launch and carried over to the second host. Ratios are of mean speeds, paired by problem, with 95% bootstrap intervals. *Speed limit*: the ceiling of Section 4 for an exact-routing system with the same slots per layer on that machine (exact optimum, that host’s highest probed rate, datasheet GPU rate, which host B’s card exceeds by 3%; Table 3 tightens it). FreeToken keeps one pooled cache and llama.cpp pins whole layers: against the pooled bound their designs allow they stand at 18–40% and 8–20%. With every weight in the VRAM of an RTX PRO 6000 (the RTX 5090’s datasheet bandwidth, 96 GB), stock llama.cpp decodes gpt-oss at 255 and Qwen3 at 178 tok/s. [†]Launch 1 for both systems: FreeToken’s backend was picked on the launch it is scored on, which favours it, and the row has no confirmation launch.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Speed limit	Ours, % of limit
Host B: Ryzen 9 9950X3D, card at 17001 MHz memory clock, CPU 72 GB/s, link 53, together 78								
gpt-oss-120b	11%	34.9	54.0 (hybrid)	69.9	1.294 [1.278, 1.312]	2.00×	172	41%
gpt-oss-120b	25%	40.0	85.6 (hybrid)	109.2	1.275 [1.254, 1.295]	2.73×	430	25%
gpt-oss-120b	40%	47.7	132.2 (offload)	152.5	1.154 [1.134, 1.172]	3.20×	519	29%
Qwen3-30B-A3B	12.5%	19.5	38.7 (hybrid)	40.0	1.032 [1.022, 1.042]	2.05×	96	42%
Qwen3-30B-A3B	25%	22.1	54.7 (hybrid)	63.1	1.153 [1.134, 1.173] [†]	2.86×	214	30%
Qwen3-30B-A3B	43.75%	28.4	103.1 (offload)	108.2	1.049 [1.036, 1.064]	3.81×	308	35%
Host S: Ryzen 9 9950X at the stock clock, card at 14001 MHz, CPU 56 GB/s, link 53, together 66; FreeToken’s backend carried over from host B								
gpt-oss-120b	11%	28.9	48.0 (hybrid)	57.9	1.207 [1.195, 1.219]	2.00×	140	41%
gpt-oss-120b	25%	33.6	78.7 (hybrid)	94.1	1.196 [1.163, 1.224]	2.80×	351	27%
gpt-oss-120b	40%	40.1	122.2 (offload)	133.6	1.093 [1.077, 1.111]	3.34×	514	26%
Qwen3-30B-A3B	12.5%	16.1	32.7 (hybrid)	33.5	1.027 [1.018, 1.036]	2.08×	78	43%
Qwen3-30B-A3B	25%	18.5	51.4 (hybrid)	53.9	1.048 [1.037, 1.059]	2.91×	174	31%
Qwen3-30B-A3B	43.75%	23.8	98.3 (offload)	95.8	0.974 [0.962, 0.987]	4.02×	305	31%

under one rule.

The instrument against its peers. Table 1 compares the three systems at six cells on two RTX 5090 hosts. On host B ours leads FreeToken at every budget, by 15–29% on gpt-oss and 3–15% on Qwen3; the lead is largest when many experts miss and the CPU path carries them (gpt-oss at 11–25%), smallest at Qwen3 12.5%, where our cache already reads host memory at 80% of the machine’s combined rate, and at Qwen3 43.75%, where few experts miss and FreeToken’s fused GPU kernels count. On host S, whose CPU reads 22% slower (56 against 72 GB/s) behind the same link, ours leads at five cells (gpt-oss 9–21%; Qwen3 12.5 and 25% 1.03 and 1.05×) and FreeToken leads at Qwen3 43.75% (0.97, interval 0.962–0.987); every ratio is 0.01–0.11 below host B’s, because with the CPU slower the law’s tables copy more of each layer’s misses and the CPU path, which is our lead, is worth less. The lead is mostly admission rationing: with LRU in place of decayed frequency, everything else equal, the cache runs 6–21% slower across the six cells of host S and trails FreeToken at four of them (the ablation, Appendix F). On FreeToken’s own headline model, Qwen3.6-35B-A3B, ours leads by 3% at one budget and ties at two; on an RTX 4090 and an RTX 3090 ours runs 1.6–2.6× llama.cpp at the cells that fit 24 GB, and on Mixtral-8x7B, with 2 of 8 experts resident, it is within 2% of llama.cpp, because there is little locality to earn (Appendix D). Teacher-forced against an all-VRAM reference, the cache’s output differs by a mean KL of 0.0019 and 0.0005 nats over the full vocabulary (top-1 agreement 98.5% and 99.3%), the same divergence

as stock llama.cpp’s own CPU placement (Appendix F).

3 A Host-Memory Law

With experts partly in host memory, a decode step does two kinds of work: GPU work that every token needs (attention, dense projections, resident experts, the output head, launches), and reads of the experts that are not resident. The second is limited by host memory. The CPU reads X_c bytes per token (the experts it runs), PCIe reads X_p bytes (the experts copied to the GPU, whether on demand or ahead of time), and when both run at once they share the DRAM:

$$T = G + \max\left(\frac{X_c}{B_c}, \frac{X_p}{B_p}, \frac{X_c + X_p}{B_{cp}}\right). \quad (1)$$

X_c and X_p come from the engine’s counters of misses, fetches and admissions, which are counts, not timings. B_c , B_p and B_{cp} come from the machine’s probe. G is one constant per engine and model, fitted once on one host (4.82 ms for our cache on gpt-oss-120b, 5.06 ms for llama.cpp) and then frozen in the repository. The form is FreeToken’s residual-bandwidth split of one layer [46], lifted to the token; the difference is that the three rates are probed rather than assumed to add, and that the law is tested as a predictor.

Tested as a predictor. On every machine of the blind test the job wrote the law’s prediction from its own probe before any model ran (Table 2; all of its cache rows are

Table 2: The law predicted blind: each machine wrote its prediction from its own bandwidth probe, with constants frozen in the public repository, before any model run. Error of predicted over measured speed, gpt-oss-120b (cache rows) and llama.cpp --n-cpu-moe (last column, %).

Host CPU	Job	n	Median err %	Max err %	llama.cpp err %
Core Ultra 7 270K (8P+16E)	069c	5	1.5	11.9	+119
EPYC 7352 (Zen 2)	069c	5	3.2	11.7	+10
EPYC 9655 (12-ch. DDR5)	069c	5	18.7	31.9	+16
Ryzen 9 7900	072	5	4.9	8.2	+4
Ryzen 9 9950X3D (A)	073	8	1.6	4.5	–
Ryzen 9 7950X	074	1	4.0	4.0	–
Ryzen 9 9950X	076	4	2.5	5.0	–
Ryzen 9 9950X3D (B)	082	0	–	–	+4, +4

gpt-oss-120b on RTX 5090 hosts). Over 29 configurations on 7 machines (33 measurements; one machine ran twice) the median error is 3.7% per configuration; without the 12-channel EPYC 9655 it is 3.3% over 24 configurations (90th percentile 7.5%). llama.cpp --n-cpu-moe is predicted within 4% on the Ryzen hosts, including two Qwen3 configurations on host B (+3.6%, +4.0%; for Qwen3, $G=4.3$ ms, the mean residual of three llama.cpp runs on another host), and misses by +10+119% on the hybrid-core and server hosts. With its constants frozen, the law scores the cache on the 3 RTX 5090 hosts rented later (24 measurements at the six cells of Table 1, not written before the runs) at a median 7.0%, 90th percentile 11.9%, worst 16% (gpt-oss-120b 40% on host S). Refit under leave-one-host-out validation, the token-level max has a median error of 3.1% (90th percentile 9.4%), the additive form $G + X_c/B_c + X_p/B_p$ 4.5% (11.3%) and a per-layer max summed over layers 3.2% (10.2%); the max beats the sum on the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, and on the EPYC 9655 rows, where both fail, the sum is closer on the two rows that copy behind a slow link, and elsewhere the two are within a few points (Appendix C). Each failure has a measured cause (Appendix C): on-demand copies behind a slow link share DRAM less evenly than the probe’s combined rate says; llama.cpp splits every matrix evenly across threads and runs at 46% of the law on a hybrid-core host; and on a 12-channel EPYC host memory stops being the limit and a per-layer latency floor takes over. G (5.17 ms refit) exceeds the whole all-in-VRAM token (3.92 ms) because it absorbs a shortfall the probe does not see: the helpers read at $1.09\text{--}1.31\times$ the probe’s time per expert, and a fitted rate factor is collinear with G at this sample size.

What the law says, and what it is for. Equation (1) is a calibrated model of execution that reads host memory in sequence with its GPU work, as our cache and llama.cpp do. For such an engine every mechanism acts only through the bytes read from host memory per to-

ken and the number of read paths in use: a second path adds bandwidth on every host of the blind test but the 12-channel EPYC (B_{cp} against $\max(B_c, B_p)$: 8–17% on its Ryzen hosts, 1% on the Core Ultra, 33% on the Zen 2 EPYC; the RTX 3090 host’s Core i9-11900KF, where the two paths together read less than the CPU alone, is a second exception), and that is the whole gain available to fetching or splitting misses between the CPU and PCIe. Prefetching never reduces the bytes; what it can buy is overlap of the reads with the step, which the law does not price and Section 5 measures (there the law over-predicts overlapped execution by up to 53%). The law’s job in the system is the split. Its per-layer form,

$$T(n, f) = \max\left(\frac{fS}{B_p} + g, [n > f](L_c + \frac{(n-f)S}{B_c}), [0 < f < n]\frac{nS}{B_{cp}}\right), \quad (2)$$

with g the GPU’s expert work per layer ($48\mu\text{s}$ for Qwen3, $37\mu\text{s}$ for gpt-oss, from profiles) and $L_c=20\mu\text{s}$ the hand-off, picks for each count n of missed experts the number f to copy with the smallest T . On host B the law’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0%; on hosts whose link we measured at 27–30 GB/s instead of 53 it fetches less and gains 21–34% on gpt-oss and 9–18% on Qwen3, the per-machine split mattering most where the link is slow (Appendix F). In the running example on host B the table is 0,0,1,1,2 ($f(0..4)$): one missed expert runs on the CPU, hidden behind the GPU’s other experts; from two misses the law copies one, and the lead over FreeToken at that cell, $1.27\times$, is largely the CPU path that FreeToken’s bandwidth model declines to use there.

4 The Speed Limit

Bound. Consider any system that executes the model’s exact routing with at most C experts of each layer resident on the GPU. Per token, each of the Lk routed experts is either served by a resident copy (the GPU reads it from VRAM) or not, in which case its S bytes are read from host memory once more: by the CPU that runs it, or by the copy that brings it into a slot. Let R^* be the reads per token of Belady’s MIN with bypass [2, 10, 50] applied per layer to the routing trace, the fewest reads of any policy, even one that knows the future (an expert may be evicted after serving the current token; a policy that keeps every requested expert until the token ends reads 0.5–4.0% more). If c experts per token run on the CPU, host memory supplies at least $\max(R^*, c)S$ bytes and the GPU reads the dense bytes D plus $(Lk - c)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq c \leq Lk} \max\left(\frac{D + (Lk - c)S}{B_{\text{gpu}}}, \frac{\max(R^*, c)S}{B_{\text{host}}}\right), \quad (3)$$

with B_{gpu} the GPU’s datasheet rate and B_{host} the highest host read rate the probe measured (87.5 GB/s on host B). A cell is *host-bound* when the minimum is at $c^*=0$,

the host term binding with every expert the GPU can take, and *GPU-bound* otherwise; on host B gpt-oss at 11 and 25% and Qwen3 at 12.5 and 25% are host-bound, the two largest budgets GPU-bound, balanced by running 13 and 30 experts per token on the CPU. The bound’s class: exact routing, whole experts of unmodified bytes, at most C of each layer resident, one token per forward pass. A system free to pool its LC slots across layers has a looser bound, 0–10% higher (Table 3); FreeToken keeps one LRU over all layers and llama.cpp pins whole layers, so Table 1 scores them against the pooled bound, and speculative decoders, which verify several tokens per pass, lie outside it. The bound assumes that an expert run on the CPU is read from DRAM (the optimum’s shortest re-read distance is 0.5 GB of host traffic, far beyond any CPU cache), that computation runs at memory speed, and that the two read paths share B_{host} freely; it drops all latencies and starts from an empty cache. B_{host} is the highest rate our probe reached, not a hardware peak, so a system that reads host memory faster than the probe could exceed the limit (a second rental of the same CPU probed 90.3 GB/s), and host B’s card, whose memory clock is $1.21\times$ the datasheet’s, read 1,847 GB/s in our probe against the 1,792 the bound assumes, which raises its GPU-bound limits by 3%. The limit exceeds the all-in-VRAM speed at the GPU-bound cells because it uses the datasheet rate, of which batch-1 kernels reach 52–61%; with the measured GPU rate (Table 3) it is 8–9% above all-in-VRAM, the CPU’s share. The traces are the models’ own greedy text on the 30 AIME-25 prompts, templated and tokenised exactly as the server does, with routing recorded teacher-forced by our llama.cpp instrumentation; replayed through a simulator of our deployed policy they give hit rates within 1.4 points of the engine’s own counters on gpt-oss and within 0.6–5.3 points on Qwen3.

Varying it, and applying it. Table 3 changes one assumption at a time. A single pool of LC slots shared across layers, the true equal-memory constraint, needs 4.5–18.6% fewer reads than per-layer slots and raises the four host-bound limits by 5–10%; it leaves the two largest budgets unchanged, where the limit is the machine’s aggregate bandwidth and read counts no longer matter. Starting the optimum warm, after 1024 tokens, changes it by at most 1.4%. B_{host} is one probe sample; the six concurrent samples on host B read 80.7 GB/s at the median, and the median lowers the host-bound limits by 8%. The GPU term is the one that moves most: at batch 1 the all-in-VRAM runs reach only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate, and with the measured rate the limit at 25% and above collapses to one aggregate line, 279 and 192 tok/s. A per-layer sum, $\sum_i \max(\text{GPU}_i, \text{host}_i)$ with the optimum’s reads per layer, which no system without cross-layer prefetch can beat, sits 1–5% of a measured token below the token-level maximum. Against the limit as published our cache stands at 25–43% on the two hosts of Table 1, FreeToken at 18–40% and llama.cpp at 8–20%

Table 3: The speed limit under each tightening, tokens per second on host B, and our cache as a percentage of the limit as published (exact optimum, highest probed host rate, datasheet GPU rate) and with every tightening at once. *no-evict*: Belady’s MIN with bypass that keeps a served expert for the step; *exact*: the same, free to evict an expert after serving it; *pool*: one pool of LC slots shared across layers; *median*: B_{host} the median of six concurrent probe samples instead of the highest; *GPU*: the GPU term from the measured all-in-VRAM time instead of the datasheet rate; *layers*: the per-layer sum, which no system without cross-layer prefetch can beat; *all*: exact, pool, median and GPU together (a warm start, not included, changes the limit by at most 1.4%).

Model	Experts on GPU	no-evict	exact	pool	median	GPU	layers	all	Ours, % of exact	% of all
gpt-oss-120b	11%	169	172	182	159	172	163	168	41	41
gpt-oss-120b	25%	427	430	474	397	279	365	278	25	39
gpt-oss-120b	40%	519	519	519	517	279	500	278	29	55
Qwen3-30B-A3B	12.5%	92	96	101	89	96	93	93	42	43
Qwen3-30B-A3B	25%	209	214	234	197	192	193	191	30	33
Qwen3-30B-A3B	43.75%	308	308	308	307	192	298	191	35	57

of the pooled bound their designs allow; with every tightening at once on host B, ours at 33–57%. In the running example the limit on host B is 430 tok/s (279 with the measured GPU rate) and the cache runs 109 tok/s, a quarter of it. Even with every weight in VRAM, our cache with every expert in its slots matches or beats stock llama.cpp ($0.997\times$ and $1.047\times$): the cache costs nothing when the model fits.

Published systems against their own limit. Rule 11 of scientific benchmarking asks for an upper bound beside every measurement [14]. We applied a bound of the same construction as Eq. (3) (Belady’s optimum with bypass per layer, CPU and PCIe paths, a maximum per layer step; `scripts/audit.py`), with each paper’s stated hardware at datasheet rates and a routing trace of its model where we have one, to 52 published batch-1 measurements from 13 sources (Appendix G lists every row with its source and plots them). The 29 trace-scored measurements reach a median 9.5% of their own speed of light (quartiles 4.9–17.8%, best 36.8%); the 11 that needed an i.i.d. routing assumption, 34.8%, an overestimate; all 52, 14.5%. The ceiling takes the host rate from the datasheet where ours takes it from the probe, and the 4 rows with every expert resident sit at a median 4.0% with nothing offloaded, so for small models at batch 1 the dropped latencies are not small; pooled, layer-pinning and speculative systems are scored against a per-layer one-token bound that is not theirs. Even so, the trace-scored measurements sit at a median tenth, and three quarters of them below a fifth, of what their machines could deliver, and the next section says what the missing part is made of.

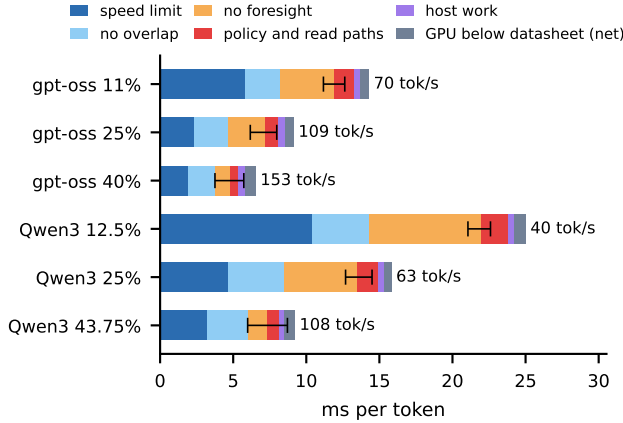


Figure 1: The accounting as a model: Shapley attribution of the gap between our cache’s measured time per token and the speed limit, for every cell of Table 1 on host B, averaged over all 120 orders in which the five fixes can be applied. Fixes: overlapping host reads with GPU work, which in Eq. (3)’s form also moves experts onto the CPU until the two paths balance; the optimum’s reads instead of the best online policy’s (foresight); the best online policy’s reads at the probe’s highest rate instead of the engine’s reads through Eq. (1) (policy and read paths); the engine’s per-token host work; and GPU kernels at the datasheet rate, the term that closes the model on the measured time. Whiskers: the range of foresight’s marginal cost over the orders.

5 Where the Seconds Go

The accounting, as a model. Between the measured time of a cell and its limit lie five modelled fixes: overlap of host reads with GPU work (in Eq. (3)’s form, which also rebalances experts onto the CPU), the optimum’s reads instead of the online policy’s (*foresight*), the best online policy’s reads at the probe’s highest rate instead of the engine’s through Eq. (1) (policy and read paths), the engine’s per-token host work, and GPU kernels at the datasheet rate. Adding them in a fixed order makes the first one added look largest, so we report each fix’s Shapley value, its average marginal cost over every order (Fig. 1), a method with a precedent in performance debugging of microservices [24]. Two of the five terms are definitions rather than measurements: the GPU term is the residual, its factor calibrated per cell so that the model with no fix applied reproduces the measured time, and “overlap” is Eq. (3)’s form. On host B the model has two regimes. In the four host-bound cells missing foresight is the largest cost, 37–53% of the gap (26–31% of the token; in the running example, 37%); in the two GPU-bound cells host reads serialised with GPU work is largest, 39–46% of the gap, and foresight second at 23%. Policy and read paths cost 12–16%, per-token host work 3–10%, and GPU

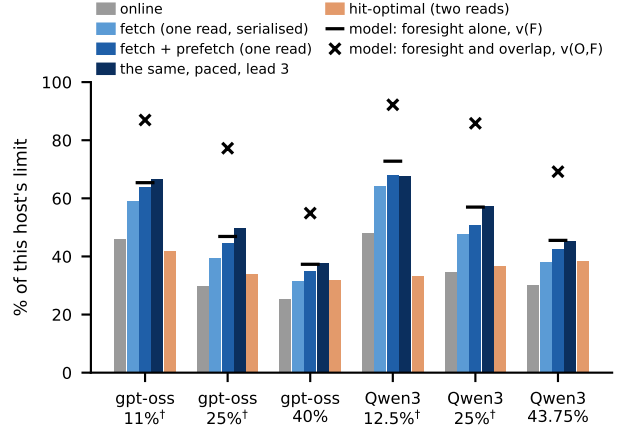


Figure 2: The accounting, measured: the engine’s speed as a percentage of the limit on oracle host O3 (RTX 5090, Ryzen 9 9950X, host memory 61.5 GB/s at best) at the six cells of Table 1, with the online policy and with oracles that read the routing of the rest of the sequence. *fetch*: MIN with bypass, each admitted miss fetched into its slot in the step that needs it (one read, serialised); *fetch + prefetch*: the same, plus a single-read prefetch of experts whose first use is at least two steps away; *paced*: the same prefetch on the paced copy path with a three-step lead; *hit-optimal*: Belady prefetch without bypass through the background admission path, unpaced, which admits experts the optimum would bypass. Marks: the model’s foresight-only state $v(F)$ and its foresight-and-overlap state $v(O, F)$ for the cell. [†]host-bound at the limit. Every ratio’s interval is within ± 0.03 (Table 8).

kernels below datasheet, net of the overlap the engine achieves, 4–16%. The regimes are a property of the host: recomputed for the three oracle hosts, whose host memory is weaker, foresight edges overlap at three of their six GPU-bound cells. The model prices foresight and overlap as nearly independent players, and the interior states it values are not realised by any online policy. The rest of this section measures them.

Foresight, measured. An oracle policy reads the routing of the rest of the sequence from a trace recorded on the same machine over the same teacher-forced text, and the engine runs it at every cell beside the online policy (Fig. 2; Table 8, Appendix E). The policies differ in how they use the foresight. *fetch* is the limit’s own policy, MIN with bypass, with its admissions served the way the bound counts them: a miss whose next use comes before the furthest next use among the residents is fetched into the victim’s slot in the step that needs it and read from host memory once; every other miss runs on the CPU. It overlaps nothing across steps, so it is the model’s foresight-only state $v(F)$ realised. *fetch + prefetch* adds the overlap that foresight makes possible: an expert whose first use

is at least two steps away is copied ahead of time into the slot of the resident whose next use is further off, so that the copy lands before the use and the expert is still read once; *paced* runs the same prefetch on the engine’s paced copy path (16 MB pieces) with a three-step lead. *hit-optimal* is the policy a prefetching cache would seem to want and the one our first oracle job ran: Belady within the window, admitting every expert the coming steps will route to, soonest first, through the background admission path (here unpaced), for one future use as readily as for ten, so it admits what the optimum bypasses and re-copies what it evicts; a miss that is served on the CPU before its copy lands is read twice. The host, O3, reads 53 GB/s by the CPU, 43 over the link and 60 with both; its limits are 121–512 tok/s on gpt-oss and 68–303 on Qwen3, and the online policy stands at 25–48% of them.

With each admitted expert read once, foresight pays at every cell. The fetch oracle reads 1.04–1.14 $\times$ the optimum’s R^* (in the running example 17 experts per token against the online policy’s 34 and $R^*=15.3$), hits within 1.1 points of what the trace predicts for it, and runs 1.29, 1.32 and 1.25 $\times$ the online policy on gpt-oss and 1.34, 1.38 and 1.26 $\times$ on Qwen3 (every interval within ± 0.02), 32–64% of the limit and 66–75% of the gain the model’s $v(F)$ predicts at the host-bound cells. The law on its own counters, with G calibrated on the online run, under-predicts its time by 17% at most (–17 to –8%): a fetched expert must land before its layer’s experts run, a per-layer latency the law, which charges bytes at the probed rate, does not price; that latency is the rest of $v(F)$. Given the same foresight two to three steps ahead, the prefetch adds the overlap: fetch + prefetch runs 1.39–1.49 $\times$ the online policy on gpt-oss and 1.41–1.48 $\times$ on Qwen3, and paced 1.45–1.66 $\times$ and 1.41–1.67 $\times$, 38–68% of the limit, 56–61% of the gap closed at the host-bound cells (in the running example 1.66 $\times$, 50% of the limit, 57% of the gap). The hit-optimal oracle on the same host is slower than every prefetching single-read variant at every cell and than the fetch oracle at the four host-bound cells (it ties fetch within a point at the two GPU-bound ones), runs 1.14 $\times$ in the running example, and *loses* at the two lowest budgets (0.91 $\times$ at gpt-oss 11%, 0.69 $\times$ at Qwen3 12.5%): its admissions alone are 1.57–2.16 $\times$ R^* , its reads 1.77–2.47 $\times$, and it drives host memory at 71–82% of the probe’s best rate at the host-bound cells, where the online policy already runs at 66–82%. Our first two oracle jobs, on hosts O1 and O2, had spent foresight only in the usual ways, as this Belady prefetch and as MIN with bypass whose admissions are copied after the CPU has served them (two reads per admitted expert), and found the first worth 25–63% and the second 13–23% at the four cells of 25% and above, and at most 6% or a loss at the two lowest (Appendix E). Both read 1.7–2.5 $\times$ R^* : the Belady prefetch’s host traffic sat at 89–94% of the probe’s best rate at every window on O1, so its speed at the host-bound cells was R^* over its reads times that utilisation, 44–45% of the limit with the whole sequence in view; and the bytes-optimal one

read 4–5% *more* than the online policy at the two lowest budgets, where it gained 0–3%. The excess reads, not the foresight, were what made foresight look worthless where host memory is saturated: admitting what the optimum bypasses, and reading an admitted expert twice, which is the admission path of every cache that serves misses on the CPU and copies them afterwards, ours included.

The accounting, measured. The four states online, fetch, fetch + prefetch and limit decompose the gap on host O3 at the host-bound cells into the bytes foresight saves (online to fetch): 34–49% of the gap; the overlap foresight allows (fetch to the faster of the two prefetch states): 8–23%; and the rest, 39–43% (Table 9; in the running example 34, 23 and 43%). The model’s shares for the same host are 52–66, 27–36 and 8–13%: it over-prices the bytes 1.3–1.5 $\times$ and the overlap 1.6–3.5 $\times$, and under-prices what remains 3.3–5.6 $\times$, which is three things the oracles leave in place. The prefetch is imprecise: its reads are 1.18–1.58 $\times$ R^* , the cost of choosing victims two to three steps early and of the misses that fall inside the lead. The CPU misses that remain are served in sequence with the step: the law over-predicts the prefetching states by +5 to +36%, least at the lowest budgets, where 19 and 46 misses per token stay on the CPU. And the GPU work runs at 52–61% of the datasheet rate the limit counts, which is most of the remaining gap at the GPU-bound cells: on Qwen3 at 43.75% on host A, with our earlier fixed FETCH table, Nsight Systems splits a token of our cache (FreeToken) into 4.93 (5.20) ms of expert copies over PCIe, 2.29 (2.21) of resident-expert products, 1.65 (1.34) of attention projections, 0.49 (0.37) of attention, 0.38 of output head and 0.97 (0.40) of norms, small kernels and cache control, with our host-side time at 0.27 ms; neither system overlaps its PCIe copies with GPU compute there, which would save 5–10%. Two things the model prices as overlap are worth nothing in this engine: turning off its within-layer overlap of a layer’s CPU misses with its GPU hits changes speed by under 1% at every cell, and predicting the next layer’s experts, the usual form of prediction [16, 54], never lowers host bytes, which is what we measured when we built it. Foresight across tokens is the lever, and the first oracle job measured its horizon: half of the Belady prefetch’s speed gain at gpt-oss 25% arrived between windows of 4 and 16 tokens (9.2 by log interpolation), a horizon of the same order as the 10 tokens at which, in the next section, single-read Belady on its trace closes half of the read gap.

6 What the Traces Say

The accounting names foresight across tokens as the largest cost where the host binds, and the engine confirms it. What would recover it, and what would not? We replay the models’ own sampled text for 9 MoE models [28, 45, 29] (9 traces of 33,000–152,000 tokens, the cache carried across

Table 4: Host reads per token relative to the exact optimum (Belady’s MIN with bypass over the whole future), median over nine models at three budgets, and the range over all 27 model–budget cells. Event-atomic replay of each model’s own sampled text; every online policy is scored on the last 90% of the trace. Mixtral (8 experts) uses $C = 2, 3$ and 4.

Policy	$C=E/8$	$E/4$	$3E/8$	range
LRU	1.44	1.77	2.03	1.37–2.26
LFU	1.44	1.93	2.40	1.37–6.15
Decayed frequency	1.45	1.70	1.94	1.35–2.10
with hysteresis κ	1.46	1.75	2.00	1.36–2.55
ARC	1.44	1.76	1.99	1.37–2.23
S3-FIFO	1.44	1.69	1.94	1.38–2.11
Static, profiled	1.92	2.74	3.48	1.43–13.24
Belady, $W=1$	1.18	1.45	1.69	1.07–2.06
Belady, $W=4$	1.01	1.13	1.31	1.00–1.95
Belady, $W=16$	1.00	1.00	1.01	1.00–1.66

conversations) through an event-atomic simulator: each token requests its k experts per layer as one set, and no expert of the current token is evicted to serve it, the accounting that cache-level studies of this problem call for [51].

Policies. Table 4 scores LRU, LFU, decayed frequency, ARC, S3-FIFO and a profiled static cache against the exact optimum and against Belady within a window of W future tokens, at three budgets per model. Three things hold across models. First, the online policies are close to one another and far from the optimum: the best of them reads 35–110% more than it, while decayed frequency is the best online policy in 12 of 27 cells and S3-FIFO in 10 (within 1% of the best in 20 and 18). Where the cache turns over every token ($C/k \leq 1.5$, 7 cells) the six online policies are within 4% of one another; from $C/k=2$ LRU reads 2–11% more than decayed frequency on eight of nine models, Qwen2-57B being the exception, and classic LFU fails at large budgets ($6.2\times$ the optimum). The literature’s disagreement on recency against frequency [34, 20, 44, 12] is consistent with this regime dependence. Second, a static cache profiled on other text reads 1.4–13.2 $\times$ the optimum, because the hot set moves: the C most requested experts of a layer change by 22–58% between consecutive windows of 100 tokens and 33–75% between windows of 1,000 (median 51%), so popularity follows the conversation; the skew is mild, the top quarter of experts taking 31–56% of requests. Third, four tokens of foresight beat every online policy in every cell (median 1.13 $\times$ the optimum at $E/4$), so foresight is worth several times the choice of policy. Our deployed hysteresis costs 1–8% reads over plain decayed frequency on eight models and up to 22% on Qwen3; it was tuned for time, where it rations copies, not for reads.

How far ahead. The optimum reads 26–56% fewer experts from host memory than our deployed decayed-frequency policy (median 41%). The lookahead needed scales with the cache’s turnover time C/k , slots per layer over experts per token: half of the gap closes at $W_{50} \approx 0.59 (C/k)^{1.33}$ tokens ($r=0.971$ in log–log over 26 points; the exponent’s 95% interval over a bootstrap of models is 1.09–1.47, its leave-one-model-out range 1.24–1.37, and without the 6 points whose W_{50} lies below one token it is 1.51), and 90% needs a median of $2.7 C/k$. Small caches ($C/k \leq 2$) recover half of the gap with one or two tokens of foresight; gpt-oss-120b at 25% ($C/k=8$), the running example, needs about 10 tokens, and at 50% about 34 (both measured on its trace; the fit gives 9 and 23). A prefetching system needs that horizon plus the lead of its copy path, two to three steps in ours. The law adds the condition: fewer reads are worth time only where host memory is the binding term, and a system that streams every expert through a saturated bus each token, as on the devices of [49, 48], has no slots to fill and nothing to foresee.

Speculation is not foresight. Speculative decoding verifies K draft tokens in one forward pass, so a layer’s experts for K positions are requested together and a missed expert is read once for all of them, which several systems propose as the source of foresight for a cache [6, 23, 31]. On the same traces, with K positions per step and drafts accepted at rate α until the first rejection, batched verification saves an online cache a median 15–32% of its reads across models at $K=8$ when rejected drafts cost nothing (48% on the model that gains most), the equivalent of a window of two to four tokens; once the rejected drafts’ experts are counted, the saving is 8–12% at $\alpha=0.9$ and becomes a loss of 18–20% at $\alpha=0.8$, because the union of K positions’ expert sets is 20–79% of Kk , several times C (Appendix B lists the variants). The optimum gains little from batching (–7 to 15%). Foresight must come from knowing future routing, as a router decoupled from the backbone [3] or a forecaster [41] would provide, and W_{50} says how far ahead.

7 Related Work

Offloading systems. Demand-paging systems cache experts per layer and copy misses over PCIe [8, 43, 35, 37, 32]. CPU-execution systems run non-resident experts on the host [19, 5, 53, 52, 40]; a set-associative GPU cache with CPU-executed misses [15], a llama.cpp fork [21, 22] and two llama.cpp pull requests [27, 7] are the closest designs to ours, and FreeToken [46] and DALI [55] split each layer’s misses between CPU and PCIe, FreeToken from a closed-form bandwidth model. Pipelined Sharding [38] plans sub-layer placement under a VRAM budget. Predictors guess upcoming experts from the previous layer’s state [16, 54], several steps ahead with a learned fore-

caster [41, 44], or a batch’s routes across layers [11]; ReadME [3] makes routing known in advance by decoupling the router and then caches in Belady’s manner. Speculative decoding has been combined with expert offloading as a source of batching [6, 23, 31]. Batched-throughput systems [33, 4, 47] and SSD tiers [25, 26] are outside our single-request, host-DRAM scope.

Models, bounds and audits. FreeToken’s split and our Eq. (2) are bandwidth models of one layer; Eq. (1) is a whole-token model tested blind across machines and against alternatives. Max-of-paths cost models go back to FlexGen [33] and MoE-Lightning [4]; WiSP [49] bounds decode by PCIe turnover, Budgeting Bytes [48] by bytes over bandwidth, and MoE-CAP [18] measures bandwidth utilisation against a single peak. Qiu et al. [30] predict llama.cpp throughput from GGUF metadata for models held entirely in GPU or unified memory; Suram [36] matches one offloaded configuration to bytes per token over bandwidth; Si et al. [34] derive when a copy pays off and find recency competitive with frequency at large budgets. The roofline [42] is the single-memory ancestor of our bound. Zhang [51] proposes a trace-level evaluation contract for expert caching, decomposes the online-to-optimal gap in cache blocks in one fixed order, and audits the reporting of ten papers; SeqMoE [41] reports four systems against the all-in-VRAM speed. Trace studies of policies cover two or three models each [51, 34, 20, 12]. Shapley attribution of a performance gap has a precedent in ShapleyIQ [24]. Limited lookahead in paging goes back to Albers [1]; Hasslinger et al. [10] combine Belady within a window with LRU or LFU beyond it for web caches, the policy family we use to measure the horizon.

8 Limitations

- **Hardware.** Consumer cards (RTX 5090 for the headline comparisons, a 4090 and a 3090 at the cells that fit 24 GB; an RTX PRO 6000 for all-in-VRAM runs) and desktop Ryzen and Intel hosts, rented; GPU clocks could not be locked, power limits differed (350–600 W), and host B’s card reads device memory 3% above the datasheet rate, where the common-clock cards read below it; Table 1 therefore reports host S beside it, where our lead is 0.01–0.11 smaller and FreeToken leads at one cell. A 12-channel server CPU breaks the law’s bandwidth assumption (Section 3). Intervals cover problem sampling on one machine, not machine-to-machine variance, which Fig. 4 shows across eight hosts; the oracle factorial ran on one host.
- **Workload.** Single-request decode; AIME prompts, which were also used while developing the system; greedy decoding; prompts of 52–879 tokens (median 130 for Qwen3, 186 for gpt-oss). Long outputs are measured on held-out problems. Prompts and batches

above one bypass the cache and run 79–83% and 77–79% of stock llama.cpp’s speed; speculative verification bypasses it and is not measured.

- **Competitor settings.** FreeToken ran its faster backend per budget with its own bandwidth calibration, default thread count and fetch cap, while our FETCH table was swept; its CPU backend failed on a hybrid-core host at its default thread count and was rerun on the performance cores. Each comparison uses one confirmation launch, except Qwen3 at 25%, which is from the selection launch for both systems.
- **The bound, the accounting and the audit.** Exact routing only: expert skipping, substitution or deferral change the model and fall outside it; a per-layer budget, which a pooled cache can beat by 5–10%. Two of the accounting’s five terms are definitions (Section 5). The oracles read a recorded trace, not a predictor; they measure what this engine does with foresight, and their prefetch still reads 1.18–1.58× the optimum’s bytes. The published rows use datasheet ceilings and, for eleven of them, i.i.d. routing.
- **Constants and outputs.** G , g and L_c are fitted once and frozen; the per-layer model overstates differences between FETCH tables about 1.8×. Output parity is measured by teacher-forced KL divergence, agreement and loss against an all-VRAM reference, not by task accuracy.

9 Conclusion

On a consumer machine, the optimum’s host bytes over the probed host rate and the datasheet GPU rate give a speed limit in seconds that published systems reach a median tenth of and ours a quarter to two fifths. A host-memory law, predicted blind and tested against its alternatives, places an engine in the limit’s coordinates and tells it which path to use for a miss on a given machine. Where the host binds, the largest piece of the gap is the policy’s ignorance of future routing, and in the engine that piece is recovered only when foresight is spent as the optimum spends it, admitting what MIN with bypass admits and reading each admitted expert once: so spent, it runs our cache 25–38% faster on the bytes alone and 42–67% faster with the overlap it allows, at every budget; spent as a Belady prefetch or through the serve-then-copy admission of its first oracles, it read 1.7–2.5× the optimum’s bytes and gained at most 6%, or lost, at the lowest budgets. What remains is the prefetch’s own excess reads, CPU misses served in sequence, and GPU kernels at half the datasheet rate. Nine models’ traces say that neither an online policy nor speculative batching buys that foresight; it must come from knowing the routing a few to tens of tokens ahead, and W_{50} says how many.

References

- [1] Susanne Albers. On the influence of lookahead in competitive paging algorithms. *Algorithmica*, 18(3):283–305, 1997.
- [2] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [3] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 116126–116148, 2024. arXiv:2410.19123.
- [4] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [5] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.
- [6] Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference. *arXiv preprint arXiv:2510.10302*, 2025.
- [7] csantiago78. llama.cpp pull request #27861: GPU-resident LRU cache for host-offloaded MoE expert weights. <https://github.com/ggml-org/llama.cpp/pull/27861>, 2026. Draft, accessed September 2026.
- [8] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [9] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv preprint arXiv:2502.12224*, 2025.
- [10] Gerhard Hasslinger, Juho Heikkinen, Konstantinos Ntougias, Frank Hasslinger, and Oliver Hohlfeld. Optimum caching versus LRU and LFU: Comparison and combined limited look-ahead strategies. In *Proceedings of the 16th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, pages 1–6, 2018.
- [11] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. Expert-Flow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling. *arXiv preprint arXiv:2410.17954*, 2024. Accepted at DAC 2026 (per arXiv v2).
- [12] Duc Hoang, Ajay Jaiswal, Mohammad Samragh, and Minsik Cho. SpecMD: A comprehensive study on speculative expert prefetching. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03921.
- [13] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [14] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [15] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [16] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.
- [17] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [18] Yinsicheng Jiang, Yao Fu, Yeqi Huang, et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- [19] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [20] Byeongju Kim, Jungwan Lee, Donghyeon Han, Hoi-Jun Yoo, and Sangyeob Kim. FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices. *arXiv preprint arXiv:2601.17063*, 2026.
- [21] leloch. MoE expert cache for llama.cpp: VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. <https://github.com/leloch/llama.cpp/tree/moe-cache-v2-pr>, 2026. Branch moe-cache-v2-pr, commit e3096b0 (6 August 2026). Proposed upstream as llama.cpp pull request #24524 and RFC discussion #24528; not merged. Accessed September 2026.
- [22] leloch. RFC: MoE expert cache, VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. llama.cpp discussion #24528, <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Posted 12 June 2026; accessed September 2026.
- [23] Shuhuai Li, Jianghao Lin, Dongdong Ge, and Yinyu Ye. MoE-SpAc: Efficient MoE inference based on speculative activation utility in heterogeneous edge scenarios. *arXiv preprint arXiv:2603.09983*, 2026.
- [24] Ye Li, Jian Tan, Bin Wu, Xiao He, and Feifei Li. ShapleyIQ: Influence quantification by Shapley values for performance debugging of microservices. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 4, pages 287–323, 2024.
- [25] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA:

- SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.
- [26] Yu Lin, Yiming Wang, Runyuan Cai, Hanze Liu, and Xiaodong Zeng. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- [27] miltos22. llama.cpp pull request #26563: Expert caching that greatly increases performance (self-contained, off by default). <https://github.com/ggml-org/llama.cpp/pull/26563>, 2026. Accessed September 2026.
- [28] Niklas Muennighoff, Luca Soldaini, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- [29] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [30] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [31] Anish Saxena, Po-An Tsai, Hritvik Taneja, Aamer Jaleel, and Moinuddin Qureshi. Cascade: Utility-driven speculative decoding for mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2506.20675 (2025).
- [32] Musa Shams. Paging the experts: A reproducible characterization of flash-backed MoE inference on iPhone. *arXiv preprint arXiv:2609.29032*, 2026.
- [33] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116, 2023. arXiv:2303.06865.
- [34] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [35] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [36] Shriniwas Ramesh Suram. Cacheable by design? training mixture-of-experts routers for locality against the edge memory-bandwidth wall: A pre-registered negative result, with a systems measurement study. *arXiv preprint arXiv:2608.18261*, 2026.
- [37] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [38] Aditya Ukarande, Deep Shekhar, Marc Blackstein, and Ram Rangan. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 8, 2026. Industry track. arXiv:2604.26334. Artifact: <https://github.com/deepshnv/pipeshard-mlsys26-ae>, tag v2.0.3-mlsys26.
- [39] Wenfeng Wang, Jiacheng Liu, Xiaofeng Hou, Xinfeng Xia, Peng Tang, Mingxuan Zhang, Chao Li, and Minyi Guo. MoE-SpeQ: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts. *arXiv preprint arXiv:2511.14102*, 2025.
- [40] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [41] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [42] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [43] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [44] Sanjali Yadav and Bahar Asgari. Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging. *arXiv preprint arXiv:2609.38090*, 2026.
- [45] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [46] Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [47] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [48] Hanhaodi Zhang. Budgeting bytes: A windowed storage roofline and dual-budget architecture ablations for storage-bound LLM decoding. *arXiv preprint arXiv:2609.04238*, 2026.
- [49] Jiamu Zhang, Liang Wu, Mayank Darbari, and Liangjie Hong. WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware. *arXiv preprint arXiv:2606.21868*, 2026.
- [50] Lei Zhang, Reza Karimi, Irfan Ahmad, and Ymir Vigfusson. Optimal data placement for heterogeneous cache, memory, and storage systems. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (PO-MACS)*, 4(1):6:1–6:27, 2020.
- [51] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [52] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference. In *Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. arXiv:2501.10375.
- [53] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.

- [54] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.
- [55] Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. DALI: A workload-aware offloading framework for efficient MoE inference on local PCs. *arXiv preprint arXiv:2602.03495*, 2026.

A Scientific benchmarking checklist

How this paper follows the twelve rules of Hoefer and Belli [14].

Rule	Where and how
1. Report the base case of a speed-up and its absolute performance.	Every ratio in Table 1 is next to both absolute speeds; llama.cpp is configured with per-layer expert offload at equal expert memory, the setting its users run.
2. Explain subsets of benchmarks or resources.	Two models chosen for size and precision (MXFP4, BF16), a third that is FreeToken’s own headline model; budgets span what fits a 32 GB card; the 43.75% budget is omitted where it does not fit. All 30 AIME-25 problems are used.
3. Arithmetic mean for costs, harmonic mean for rates.	Every problem decodes the same number of tokens, so the harmonic mean of rates is total tokens over total time; ratios of total time agree with the reported ratios of mean rates within 0.003.
4. Avoid summarising ratios.	Ratios are computed from mean speeds, never averaged across budgets or models; ranges list the per-budget values.
5. Report whether measurements are deterministic; give confidence intervals.	Decode is greedy and deterministic per engine; timing is not. Every comparison has a 95% interval.
6. Do not assume normality.	Intervals are bootstrap percentile intervals (10,000 resamples, paired by problem); no distributional assumption.
7. Compare nondeterministic data soundly.	Paired bootstrap of the ratio of means; the variant a system is scored with is picked on a separate launch (except the Qwen3 25% row of Table 1).
8. Check that central tendency is the right summary.	Mean speed over 256 tokens is FreeToken’s own metric; per-token p50 and p99 latencies are recorded for every problem in the released rows.
9. Document all factors and the setup.	Machines, probes, power limits, memory clocks (Section 2), software commits (llama.cpp 4da6337, FreeToken 0d652e7), flags and job scripts are released; Appendix H.
10. Report measurement, synchronisation and summarisation.	One client times every system from streamed token arrivals; no server-side timers enter a comparison (Section 2).
11. Show upper performance bounds.	The speed limit (Section 4) and all-in-VRAM speed are columns of Table 1; Table 3 tightens the limit four ways; Section 4 applies it to the published literature.
12. Plot enough to interpret; connect points only for trends.	Figures 1, 2 and 6 are bar charts; Figs. 3, 4 and 7 are scatter plots; Fig. 5 connects measured windows for the trend only, and its W_{50} is a stated log interpolation between two measured windows.

B Prediction log

Each job’s predictions are in the header of its script on the public `gpu` branch, committed and pushed before the machine started; git history orders every prediction before its measurement. Outcome notes with the statistics are in `prereg/`. Table 6, generated from `prereg/scorecard.clauses.json`, splits every prediction of jobs 073–095 into its separable clauses and scores each under one rule: *held* only when the point estimate is on the predicted side and the 95% paired-bootstrap interval excludes the threshold (for a band, lies inside it); *held (point)* when only the point estimate is, or no interval exists; *failed* when the point estimate is on the wrong side; *untested* when the clause could not be evaluated; *void* when the job was voided. Of 299 clauses, 156 held, 28 held on the point estimate, 103 failed, 8 were untested and 4 void: 7 of 19 scored clauses held in jobs 073–075, when the system was being built, 19 of 23 in jobs 076–081, 11 of them sign-only predictions made after a same-host pilot, 24 of 38 in jobs 082–087, and 106 of 207 in jobs 088 onward, where 30 of 36 sign clauses held (33 on the point estimate) and 68 of 135 bands failed. Most bands that failed are the sizes of effects whose direction held: our slow-link lead landed at $1.51\text{--}1.58\times$ against a predicted $1.10\text{--}1.50$; the rerun of Table 1 on a second host (job 089) gave ratios $0.06\text{--}0.11$ below host B’s at five of six cells against a predicted ± 0.06 , host-bound speeds $14\text{--}17\%$ lower against a predicted $\pm 12\%$, and LRU $6\text{--}8\%$ slower than decayed frequency at the two GPU-heavy cells against a predicted $10\text{--}30\%$; on the 4090 and 3090 hosts (jobs 091, 092; Appendix D) our lead over FreeToken and its fraction of the limit overshoot their bands, FreeToken’s carried-over

backend fell far below its band, and two launch-order checks exceeded 3%; in the first oracle job (093; Section 5) the oracle’s gain fell outside 20–80% at the two lowest budgets (it lost), its speed fell just below the predicted 45–65% of the limit at all four host-bound cells (43.7–44.8%), its hit rate and admissions missed the optimum’s by more than the predicted 3 points and 30% at three and two of the four, no-overlap cost 0% against a predicted 5–25%, and all-CPU cost 15% on Qwen3 against a predicted 0–10%; in the bytes-oracle job (094) the bands missed as Appendix E describes; in the single-read job (095) 44 of 56 clauses held, and the 12 that failed are of size: the law on the fetch oracle’s counters was predicted within 10% and missed by 13–17% at four cells, the fetch oracle’s reads were predicted within 10% of the simulation’s and came 17% below at the two GPU-bound cells (the engine’s cache was warm from the previous configuration), the prefetching oracle was predicted at 55–85% of the limit at the host-bound cells and reached 44 and 51% at the two 25% cells, the law’s over-prediction of it was predicted at 15–50% and came to 7 and 13% at the two lowest budgets, and the paced variant, predicted within 10% of the unpaced one, was 12–13% faster at the two 25% cells. The 11 threshold failures are ours at $1.56\text{--}1.78\times$ llama.cpp on the slow hosts against a predicted 1.8, the Mixtral tie against a predicted $1.3\times$, the hit-optimal oracle’s gain at the two GPU-bound cells (48 and 56%) against a predicted ceiling of 15%, the bytes-optimal oracle short of beating the hit-optimal one by 15% at the two lowest budgets, and its 16-token window short of keeping 70% of its gain at three cells; the 2 equality failures are the 4090 host’s FETCH tables, which copy one more expert than predicted; the 3 sign failures are LRU behind FreeToken at gpt-oss 11%, where we predicted it would still lead, and the hit-optimal oracle slower than the online policy at gpt-oss 11% and Qwen3 12.5%. Against the running log kept during the study, two clauses are downgraded (a llama.cpp clause of job 074b that the log counted as held was never run; job 085b’s own restatement of its fourth prediction, “at least 2 of 3”, failed at 1 of 3, though the pooled 4 of 6 held), job 073’s “identical outputs” is scored on fully identical outputs (0–3%) rather than on agreement of the first 160 characters (27–47%), and ten clauses the log counted as held, among them job 087’s first, hold on the point estimate only under the interval rule; `prereg/scorecard.outcome.md` lists every change. Job 085’s Qwen3 half ran as 085b on a second machine with a slow link after a hung conversion. Job 084 ran on a throttled card and was rerun, with its predictions unchanged, as 084b/084c. The batched-verification variants of Section 6 (rejected drafts free, routed like the true token, or routed at random) are in `prereg/batchk.outcome.md`.

Table 6: Prediction scorecard, jobs 073–095: every committed prediction split into its separable clauses and scored under one rule. *Held*: the point estimate is on the predicted side and the 95% paired-bootstrap interval excludes the threshold (for a band, lies inside it). *Held (point)*: the point estimate is on the predicted side but the interval includes the threshold, or no interval exists. *Untested*: the clause could not be evaluated. *Void*: the job was voided (084, a throttled card; its predictions are scored on 084b). Types: sign (a direction), threshold (a numeric bar), band (an interval), equality (a deterministic outcome). Measured values are ratios of mean speeds with 95% intervals unless the clause says otherwise; `prereg/scorecard.clauses.json` names the source of every number. Totals: 299 clauses, 156 held, 28 held (point), 103 failed, 8 untested, 4 void.

Job	Clause	Type	Measured	Status
Jobs 073–075: 20 clauses, 7 held, 3 held (point), 9 failed, 1 untested, 0 void				
073 P1a	ours+FETCH $\geq 5\%$ ahead of FreeToken, 11%	threshold	1.109 [1.098, 1.120]	held
073 P1b	ours+FETCH $\geq 5\%$ ahead of FreeToken, 25%	threshold	1.084 [1.076, 1.092]	held
073 P1c	ours+FETCH within $\pm 5\%$ of FreeToken, 40%	band	0.941 [0.933, 0.951]	failed
073 P2	ours+FETCH $\geq 1.8\times$ llama.cpp, every budget	threshold	1.832 [1.808, 1.856]; 2.358 [2.322, 2.396]; 2.647 [2.605, 2.686]	held
073 P3a	same-problem warm-up reads 5–15% faster, the two caches	band	0.991 [0.962, 1.028]; 0.997 [0.986, 1.008]	failed
073 P3b	same-problem warm-up within $\pm 2\%$, llama.cpp	band	0.962 [0.934, 0.979]	failed
073 P4	$\geq 80\%$ of greedy outputs identical to llama.cpp’s, every system	equality	ours (6 configurations): 0.000–0.033; FreeToken (6 configurations): 0.000; llama.cpp -ncmoe 32 / 22: 0.700 / 0.400	failed
074 P1	ec-bench launch time ≤ 0.25 ms, C51	threshold	0.201	held (point)
074 P2a	ours v2 ≥ 127 tok/s, C51	threshold	95.1	failed
074 P2b	ours v2 ahead of FreeToken offload, C51	sign	0.831 [0.817, 0.845]	failed
074 P3a	ours v2 ≥ 59 tok/s, C14	threshold	41.5	failed
074 P3b	ours v2 ≥ 91 tok/s, C32	threshold	65.3	failed
074 P4	half-life 64 within $\pm 2\%$ of half-life 16, C51	band	1.012 [0.997, 1.027]	held (point)
074b P1	GPU sampling cuts between-step host time by ≥ 1 ms	threshold	1.6	held (point)
074b P2a	ours v2 $\geq 10\%$ faster with GPU sampling, C51	threshold	1.117 [1.109, 1.125]	held
074b P2b	stock llama.cpp $\geq 3\%$ faster with GPU sampling	threshold	–	<i>untested</i>

continued

Job	Clause	Type	Measured	Status
074b P3	ours v2+FETCH with GPU sampling $\geq$ FreeToken offload, C51	sign	1.074 [1.063, 1.085]	held
075 P1	ours ahead of FreeToken at 40% (RTX PRO 6000)	sign	1.122 [1.107, 1.137]	held
075 P2	ours within -3 to +10% of FreeToken at 60%	band	1.118 [1.109, 1.127]	failed
075 P3	ours at 60% $\geq$ 55% of the all-in-VRAM speed	threshold	0.726 [0.717, 0.735]	held
Jobs 076–081: 23 clauses, 19 held, 4 held (point), 0 failed, 0 untested, 0 void				
076 P1	ours ahead of FreeToken at 11/25/40%	sign	1.205 [1.190, 1.220]; 1.217 [1.201, 1.234]; 1.118 [1.108, 1.129]	held
076 P2	ours $\geq 1.8\times$ llama.cpp, every budget	threshold	1.924 [1.894, 1.956]; 2.683 [2.631, 2.735]; 3.191 [3.130, 3.250]	held
076 P3	the law within 8% on own-text runs (four configurations)	band	1.009 [0.962, 1.066]; 1.006 [0.947, 1.086]; 1.052 [0.960, 1.164]; 0.960 [0.909, 1.024]	held (point)
077 P1a	both entrants run end to end on gpt-oss-120b	equality	30 of 30 problems at every budget for both	held
077 P1b	both entrants faster than llama.cpp at 25/40%	sign	pipeshard 25 / 40%: 1.348 [1.336, 1.360]; 1.417 [1.399, 1.435]; leloch 25 / 40%: 1.446 [1.431, 1.461]; 1.280 [1.269, 1.291]	held
077 P2	ours ahead of both entrants, every budget	sign	1.421 [1.403, 1.439]; 1.896 [1.871, 1.921]; 2.343 [2.293, 2.391]	held
078 P1	ours ahead of FreeToken at 12.5/25% (Qwen3)	sign	1.071 [1.063, 1.080]; 1.067 [1.043, 1.094]	held
078 P2	ours ahead of KTransformers, every budget	sign	2.168 [2.140, 2.197]; 3.059 [2.975, 3.162]; 4.420 [4.310, 4.550]	held
078 P3	ours $\geq 1.5\times$ llama.cpp, every budget	threshold	2.122 [2.092, 2.151]; 3.051 [2.967, 3.152]; 4.470 [4.350, 4.609]	held
079 P1a	host-side time per step < 1 ms	threshold	0.266	held (point)
079 P1b	host-side time $< 10\%$ of the token time	threshold	0.027	held (point)
079 P2	$\geq 1.5\times$ FreeToken’s kernels per token	threshold	1.606	held (point)
079 P3	ours within $\pm 5\%$ of FreeToken at 43.75%	band	0.991 [0.975, 1.009]	held
080 P1	law table $\geq 3\%$ over the current table, 43.75%	threshold	1.048 [1.041, 1.055]	held
080 P2	law table within 2% of the sweep’s best, 43.75%	band	0.991 [0.986, 0.996]	held
080 P3	best table leads FreeToken, 43.75%	sign	1.049 [1.036, 1.064]	held
080 P4	law table beats the current table, 25%	sign	1.067 [1.062, 1.071]	held
081 P1	law table beats current, gpt-oss, every budget	sign	1.079 [1.072, 1.085]; 1.054 [1.041, 1.066]; 1.033 [1.021, 1.044]	held
081 P2	ours leads FreeToken, gpt-oss 11/25/40%	sign	1.294 [1.278, 1.312]; 1.275 [1.254, 1.295]; 1.154 [1.134, 1.172]	held
081 P3a	law table beats current, Qwen3 12.5%	sign	1.080 [1.073, 1.084]	held
081 P3b	ours leads FreeToken, Qwen3 12.5%	sign	1.032 [1.022, 1.042]	held
081 P4a	ours $\geq 1.8\times$ llama.cpp, gpt-oss, every budget	threshold	2.001 [1.964, 2.041]; 2.731 [2.668, 2.795]; 3.196 [3.133, 3.259]	held
081 P4b	ours $\geq 2\times$ llama.cpp, Qwen3 12.5%	threshold	2.051 [2.026, 2.077]	held
Jobs 082–087: 43 clauses, 24 held, 4 held (point), 10 failed, 1 untested, 4 void				
082 P1	steady-state lead over FreeToken, gpt-oss, held-out prompts	sign	–	untested
082 P2	FreeToken gains more after 256 tokens than ours, every budget	sign	FreeToken: 1.017 / 1.065 / 1.071; ours: 1.038 / 1.094 / 1.117	failed
082 P3	FreeToken ahead in steady state, Qwen3 43.75%	sign	1.054 [1.025, 1.086]	failed
082 P4a	every ablation step $\geq -1\%$	threshold	0.723 [0.712, 0.733]; 0.744 [0.740, 0.748]	failed
082 P4b	ours $\geq 2\times$ stock at 25%, both models	threshold	2.658 [2.566, 2.749]; 2.800 [2.743, 2.858]	held
082 P5a	parity: $\geq 98\%$ top-1 agreement, both models	threshold	0.986 / 0.993	held
082 P5b	parity: $ \Delta\text{NLL} \leq 1\%$, both models	threshold	-0.0022 / 0.0031	held
082 P6	llama.cpp Qwen3 25/43.75% within 5% of the law	band	0.966 [0.962, 0.969]; 0.962 [0.958, 0.965]	held
083 P1	still reasoning at 2,048 tokens on ≥ 8 of 10, both systems	threshold	ours: 9 / 9 / 10; FreeToken: 10 / 9 / 10	held
083 P2	ours within 15% of its Table 1 rate	band	1.046 [1.019, 1.072]; 1.037 [1.013, 1.060]; 1.033 [1.003, 1.061]	held
083 P3a	ours leads FreeToken, all tokens, 11/25/40%	sign	1.358 [1.338, 1.378]; 1.322 [1.303, 1.342]; 1.191 [1.172, 1.212]	held
083 P3b	ours leads FreeToken, tokens 257-2,048	sign	1.366 [1.343, 1.389]; 1.328 [1.310, 1.350]; 1.194 [1.175, 1.215]	held
083 P4	lead $\geq 15\%$ at 11 and 25%	threshold	1.358 [1.338, 1.378]; 1.322 [1.303, 1.342]	held
084 P1	Qwen3 all in VRAM at 120-190 tok/s	band	–	void
084 P2	gpt-oss all in VRAM within 5% of 261.4 tok/s	band	–	void
084 P3	ours with 128 slots within 5% of all in VRAM, both	band	–	void
084 P4	sim. hit rate within 5 points; ours at 20-50% of the limit	band	–	void
084b P1	Qwen3 all in VRAM at 120-190 tok/s	band	177.8 [177.1, 178.4]	held
084b P2	gpt-oss all in VRAM within 5% of 261.4 tok/s	band	0.977 [0.971, 0.982]	held

continued

Job	Clause	Type	Measured	Status
084b P3	ours with 128 slots within 5% of all in VRAM, both	band	0.997 [0.991, 1.004]; 1.047 [1.043, 1.052]	held (point)
084b P4a	sim. hit rate within 5 points of the engine's, every budget	band	-0.014 / -0.008 / -0.0003 / -0.053 / -0.026 / -0.006	failed
084b P4b	ours at 20-50% of the speed limit, every budget	band	0.413 / 0.256 / 0.294 / 0.433 / 0.302 / 0.352	held (point)
084c	<i>No predictions of its own: it supplies the gpt-oss routing trace that 084b-P4a and 084b-P4b are scored on.</i>	—	—	—
085 P1	law's tables fetch less than on the headline machine	equality	gpt-oss: 0,0,0,1,1 (two entries lower); Qwen3: 0,0,0,1,1,2,2,3,3 (six entries lower)	held
085 P2a	law beats fixed, gpt-oss, every budget	sign	1.340 [1.327, 1.355]; 1.274 [1.256, 1.292]; 1.211 [1.187, 1.235]	held
085 P2b	law's gain larger than headline at ≥ 2 of 3 (pro rata of 4 of 6)	threshold	3 of 3 (every interval excludes the headline value); pooled with 085b: 6 of 6	held
085 P3	ours leads FreeToken, gpt-oss, every budget	sign	2.030 [1.993, 2.068]; 1.699 [1.662, 1.737]; 1.428 [1.391, 1.463]	held
085 P4	lead larger than headline at ≥ 2 of 3 (pro rata of 4 of 6)	threshold	3 of 3 (every interval excludes the headline value); pooled with 085b (1 of 3): 4 of 6	held
085 P5	llama.cpp at 25% within 5% of the law, gpt-oss	band	0.818 [0.817, 0.819]	failed
085b P1	law's table fetches less than on the headline machine	equality	0,0,1,1,2,2,3,3,4 (three entries lower)	held
085b P2a	law beats fixed, Qwen3, every budget	sign	1.184 [1.177, 1.190]; 1.149 [1.142, 1.157]; 1.094 [1.086, 1.101]	held
085b P2b	law's gain larger than headline at ≥ 2 of 3	threshold	3 of 3 (every interval excludes the headline value)	held
085b P3	ours leads FreeToken, Qwen3, every budget	sign	1.067 [1.058, 1.076]; 1.076 [1.063, 1.089]; 1.038 [1.017, 1.060]	held
085b P4	lead larger than headline at ≥ 2 of 3	threshold	1 of 3: 12.5% 1.067 [1.058, 1.076] > 1.032; 25% 1.076 [1.063, 1.089] < 1.153; 43.75% 1.038 [1.017, 1.060] below 1.049 with the interval covering it	failed
085b P5	llama.cpp at 25% within 5% of the law, Qwen3	band	0.974 [0.972, 0.976]	held
086 P1	FreeToken as shipped within 15% of its paper's rate	band	93.8 [91.3, 96.5]	held (point)
086 P2	ours leads FreeToken at 12.5/25/37.5%	sign	1.030 [1.023, 1.038]; 1.010 [0.998, 1.021]; 1.003 [0.994, 1.013]	failed
086 P3	ours $\geq 2\times$ llama.cpp, every budget	threshold	1.976 [1.950, 2.005]; 2.311 [2.264, 2.361]; 2.481 [2.441, 2.521]	failed
086 P4	law beats fixed, every budget	sign	1.071 [1.068, 1.074]; 1.060 [1.057, 1.063]; 1.057 [1.052, 1.061]	held
087 P1	profiled static cache $\geq 1.3\times$ stock, both models	threshold	1.307 [1.236, 1.379]; 1.889 [1.803, 1.969]	held (point)
087 P2	replacement adds less than placement, both models	sign	0.268 [0.122, 0.434]; -0.765 [-0.898, -0.619]	failed
087 P3	decayed frequency beats the profiled static cache, both	sign	1.575 [1.497, 1.674]; 1.124 [1.070, 1.187]	held
087 P4a	every step after the static cache $\geq -1\%$	threshold	0.877 [0.835, 0.924]	failed
087 P4b	ours $\geq 2\times$ stock, both models	threshold	2.680 [2.595, 2.763]; 2.815 [2.754, 2.877]	held
Jobs 088–095: 213 clauses, 106 held, 17 held (point), 84 failed, 6 untested, 0 void				
088 P1	FreeToken hybrid on 8 threads $\geq 1.3\times$ job 085's hybrid, every budget	threshold	3.12 / 2.04 / 1.4	held (point)
088 P2	FreeToken's best improves on job 085's best at ≥ 2 of 3 budgets	threshold	1.52 / 1.29 / 1.08	held (point)
088 P3a	ours leads FreeToken's best, 11% (CI > 1)	sign	1.582 [1.564, 1.603]	held
088 P3b	ours leads FreeToken's best, 25% (CI > 1)	sign	1.519 [1.472, 1.559]	held
088 P3c	ours leads FreeToken's best, 40% (CI > 1)	sign	1.506 [1.476, 1.537]	held
088 P3d	ours / FreeToken between 1.10 $\times$ and 1.50 $\times$, 11%	band	1.582 [1.564, 1.603]	failed
088 P3e	ours / FreeToken between 1.10 $\times$ and 1.50 $\times$, 25%	band	1.519 [1.472, 1.559]	failed
088 P3f	ours / FreeToken between 1.10 $\times$ and 1.50 $\times$, 40%	band	1.506 [1.476, 1.537]	failed
088 P4a	llama.cpp -t 8 within 10% of the law	band	-9.4	held (point)
088 P4b	llama.cpp -t 8 faster than -t 24	sign	1.219 [1.213, 1.225]	held
088 P5	the law's table is 0,0,0,1,1 or fetches less	equality	0,0,0,1,1	held
089 P1a	the card reports a 14,001 MHz memory clock	equality	14001 MHz	held
089 P1b	device read $\geq 1,500$ GB/s (not throttled)	threshold	1694	held
089 P2a	ours / FreeToken within ± 0.06 of Table 1, gpt-oss 11%	band	-0.088 [-0.099, -0.075]	failed

continued

Job	Clause	Type	Measured	Status
089 P2b	ours / FreeToken within ± 0.06 of Table 1, gpt-oss 25%	band	-0.078 [-0.112, -0.051]	failed
089 P2c	ours / FreeToken within ± 0.06 of Table 1, gpt-oss 40%	band	-0.061 [-0.077, -0.043]	failed
089 P2d	ours / FreeToken within ± 0.06 of Table 1, Qwen3 12.5%	band	-0.006 [-0.014, 0.004]	held
089 P2e	ours / FreeToken within ± 0.06 of Table 1, Qwen3 25%	band	-0.105 [-0.116, -0.094]	failed
089 P2f	ours / FreeToken within ± 0.06 of Table 1, Qwen3 43.75%	band	-0.075 [-0.087, -0.062]	failed
089 P3a	ours' speed within $\pm 12\%$ of host B's, gpt-oss 11%	band	-17.1	failed
089 P3b	ours' speed within $\pm 12\%$ of host B's, gpt-oss 25%	band	-13.8	failed
089 P3c	ours' speed 5-20% below host B's, gpt-oss 40%	band	-12.4	held (point)
089 P3d	ours' speed within $\pm 12\%$ of host B's, Qwen3 12.5%	band	-16.0	failed
089 P3e	ours' speed within $\pm 12\%$ of host B's, Qwen3 25%	band	-14.6	failed
089 P3f	ours' speed 5-20% below host B's, Qwen3 43.75%	band	-11.5	held (point)
089 P4	ours-first and FreeToken-first orders agree within 3%, every cell	band	-1.0 / 0.2 / 0.7 / 0.3 / -0.2 / 0.1	held (point)
089 P5a	LRU 10-30% slower than decayed frequency, gpt-oss 11%	band	18.5 [17.7, 19.3]	held
089 P5b	LRU 10-30% slower than decayed frequency, gpt-oss 25%	band	12.8 [11.5, 14.1]	held
089 P5c	LRU 10-30% slower than decayed frequency, gpt-oss 40%	band	7.8 [6.5, 9.1]	failed
089 P5d	LRU 10-30% slower than decayed frequency, Qwen3 12.5%	band	20.9 [20.6, 21.3]	held
089 P5e	LRU 10-30% slower than decayed frequency, Qwen3 25%	band	15.4 [14.9, 15.9]	held
089 P5f	LRU 10-30% slower than decayed frequency, Qwen3 43.75%	band	6.4 [5.9, 6.9]	failed
089 P5g	LRU trails FreeToken at Qwen3 12.5%	sign	0.814 [0.808, 0.821]	held
089 P5h	LRU still leads FreeToken at gpt-oss 11%	sign	0.973 [0.962, 0.986]	failed
089 P5i	LRU still leads FreeToken at gpt-oss 25%	sign	1.045 [1.016, 1.068]	held
089 P6a	prefill at 2,048 tokens within $\pm 20\%$ of stock, gpt-oss	band	0.800	held (point)
089 P6b	prefill at 2,048 tokens within $\pm 20\%$ of stock, Qwen3	band	0.825	held (point)
089 P6c	decode at 2 and 4 sequences below stock's, gpt-oss	sign	0.778 / 0.778 / 0.773 / 0.784	held (point)
089 P6d	decode at 2 and 4 sequences below stock's, Qwen3	sign	0.779 / 0.789 / 0.775 / 0.792	held (point)
090 P1a	mean KL ≤ 0.01 nats, gpt-oss	threshold	0.0019	held
090 P1b	mean KL ≤ 0.01 nats, Qwen3	threshold	0.0005	held
090 P1c	99.9th-percentile KL ≤ 0.5 nats, gpt-oss	threshold	0.121	held
090 P1d	99.9th-percentile KL ≤ 0.5 nats, Qwen3	threshold	0.059	held
090 P2a	top-1 agreement $\geq 98\%$, gpt-oss	threshold	98.5	held
090 P2b	top-1 agreement $\geq 98\%$, Qwen3	threshold	99.3	held
090 P2c	stock's -n-cpu-moe agreement within 1 point of ours, gpt-oss	band	0.33	held
090 P2d	stock's -n-cpu-moe agreement within 1 point of ours, Qwen3	band	-0.07	held
090 P3a	mean NLL within $\pm 0.5\%$ of the reference, gpt-oss	band	0.16	held
090 P3b	mean NLL within $\pm 0.5\%$ of the reference, Qwen3	band	0.19	held
091 P1a	ours leads FreeToken, gpt-oss 11% (CI > 1)	sign	1.371 [1.346, 1.395]	held
091 P1b	ours leads FreeToken, gpt-oss 25% (CI > 1)	sign	1.296 [1.265, 1.326]	held
091 P1c	ours leads FreeToken, Qwen3 12.5% (CI > 1)	sign	1.033 [1.021, 1.045]	held
091 P1d	ours leads FreeToken, Qwen3 25% (CI > 1)	sign	1.045 [1.028, 1.063]	held
091 P1e	lead in 10-35%, gpt-oss 11%	band	37.1 [34.6, 39.5]	failed
091 P1f	lead in 10-35%, gpt-oss 25%	band	29.6 [26.5, 32.6]	held
091 P1g	lead in 0-15%, Qwen3 12.5%	band	3.3 [2.1, 4.5]	held
091 P1h	lead in 0-15%, Qwen3 25%	band	4.5 [2.8, 6.3]	held
091 P2a	ours $\geq 1.8 \times$ llama.cpp, gpt-oss 11%	threshold	1.810 [1.760, 1.860]	held (point)
091 P2b	ours $\geq 1.8 \times$ llama.cpp, gpt-oss 25%	threshold	2.620 [2.520, 2.740]	held
091 P2c	ours $\geq 1.8 \times$ llama.cpp, Qwen3 12.5%	threshold	1.780 [1.760, 1.810]	failed
091 P2d	ours $\geq 1.8 \times$ llama.cpp, Qwen3 25%	threshold	2.540 [2.490, 2.610]	held
091 P3a	the law's gpt-oss table fetches no more than the headline machine's	equality	0,0,1,2,2	failed
091 P3b	the law's Qwen3 table fetches no more than the headline machine's	equality	0,0,1,1,2,3,4,4,5	failed
091 P4a	ours at 25-45% of the limit, gpt-oss 11%	band	44.4 [43.3, 45.5]	held (point)

continued

Job	Clause	Type	Measured	Status
091 P4b	ours at 25-45% of the limit, gpt-oss 25%	band	29.9 [29.1, 30.8]	held
091 P4c	ours at 25-45% of the limit, Qwen3 12.5%	band	45.7 [45.1, 46.3]	failed
091 P4d	ours at 25-45% of the limit, Qwen3 25%	band	34.3 [33.5, 35.2]	held
091 P5	ours-first and FreeToken-first orders agree within 3%, every cell	band	1.6 / 2.0 / -0.6 / 0.1	held (point)
092 P1a	ours leads FreeToken, gpt-oss 11% (CI > 1)	sign	4.460 [4.250, 4.690]	held
092 P1b	ours leads FreeToken, Qwen3 12.5% (CI > 1)	sign	2.120 [2.020, 2.210]	held
092 P1c	ours leads FreeToken, Qwen3 25% (CI > 1)	sign	2.680 [2.530, 2.830]	held
092 P1d	lead in 10-35%, gpt-oss 11%	band	346 [325, 369]	failed
092 P1e	lead in 0-15%, Qwen3 12.5%	band	112 [102, 121]	failed
092 P1f	lead in 0-15%, Qwen3 25%	band	168 [153, 183]	failed
092 P2a	ours $\geq 1.8 \times$ llama.cpp, gpt-oss 11%	threshold	1.640 [1.600, 1.670]	failed
092 P2b	ours $\geq 1.8 \times$ llama.cpp, Qwen3 12.5%	threshold	1.560 [1.480, 1.630]	failed
092 P2c	ours $\geq 1.8 \times$ llama.cpp, Qwen3 25%	threshold	2.180 [2.090, 2.270]	held
092 P3a	ours $\geq 1.3 \times$ llama.cpp on Mixtral-8x7B, C = 2	threshold	1.015 [1.008, 1.024]	failed
092 P3b	ours $\geq 1.3 \times$ llama.cpp on Mixtral-8x7B, C = 4	threshold	–	<i>untested</i>
092 P4a	the law’s gpt-oss table fetches no more than the headline machine’s	equality	0,0,0,0,0	held
092 P4b	the law’s Qwen3 table fetches no more than the headline machine’s	equality	0,0,0,0,0,0,0,0,0	held
092 P5a	ours-first and FreeToken-first orders agree within 3%, Qwen3 12.5%	band	7.0	failed
092 P5b	ours-first and FreeToken-first orders agree within 3%, Qwen3 25%	band	2.6	held (point)
093 P1a	oracle (W = all) faster than base, gpt-oss 11%	sign	0.923 [0.914, 0.933]	failed
093 P1b	oracle (W = all) faster than base, gpt-oss 25%	sign	1.398 [1.384, 1.412]	held
093 P1c	oracle (W = all) faster than base, Qwen3 12.5%	sign	0.916 [0.911, 0.922]	failed
093 P1d	oracle (W = all) faster than base, Qwen3 25%	sign	1.247 [1.235, 1.259]	held
093 P2a	oracle gain 20-80%, gpt-oss 11%	band	-7.7 [-8.6, -6.7]	failed
093 P2b	oracle gain 20-80%, gpt-oss 25%	band	39.8 [38.4, 41.2]	held
093 P2c	oracle gain 20-80%, Qwen3 12.5%	band	-8.4 [-8.9, -7.8]	failed
093 P2d	oracle gain 20-80%, Qwen3 25%	band	24.7 [23.5, 25.9]	held
093 P3a	W = 16 captures $\geq$ half of the W = 0 gain, gpt-oss 11%	threshold	–	<i>untested</i>
093 P3b	W = 16 captures $\geq$ half of the W = 0 gain, gpt-oss 25%	threshold	87	held (point)
093 P3c	W = 16 captures $\geq$ half of the W = 0 gain, Qwen3 12.5%	threshold	–	<i>untested</i>
093 P3d	W = 16 captures $\geq$ half of the W = 0 gain, Qwen3 25%	threshold	100	held (point)
093 P3e	W = 4 captures $\geq$ half of the W = 0 gain, gpt-oss 11%	threshold	–	<i>untested</i>
093 P3f	W = 4 captures $\geq$ half of the W = 0 gain, Qwen3 12.5%	threshold	–	<i>untested</i>
093 P4a	ours with the oracle at 45-65% of the limit, gpt-oss 11%	band	44.0 [43.2, 44.7]	failed
093 P4b	ours with the oracle at 45-65% of the limit, gpt-oss 25%	band	43.7 [42.5, 44.8]	failed
093 P4c	ours with the oracle at 45-65% of the limit, Qwen3 12.5%	band	44.8 [44.1, 45.5]	failed
093 P4d	ours with the oracle at 45-65% of the limit, Qwen3 25%	band	44.5 [43.3, 45.8]	failed
093 P5a	oracle hit rate within 3 points of the optimum’s, gpt-oss 11%	band	-5.9	failed
093 P5b	oracle hit rate within 3 points of the optimum’s, gpt-oss 25%	band	3.1	failed
093 P5c	oracle hit rate within 3 points of the optimum’s, Qwen3 12.5%	band	-9.2	failed
093 P5d	oracle hit rate within 3 points of the optimum’s, Qwen3 25%	band	0.4	held
093 P5e	oracle admissions within 30% of the optimum’s reads, gpt-oss 11%	band	-9	held
093 P5f	oracle admissions within 30% of the optimum’s reads, gpt-oss 25%	band	31	failed
093 P5g	oracle admissions within 30% of the optimum’s reads, Qwen3 12.5%	band	-35	failed
093 P5h	oracle admissions within 30% of the optimum’s reads, Qwen3 25%	band	7	held
093 P6a	no-overlap 5-25% slower than base, gpt-oss 11%	band	-0.5 [-1.0, -0.1]	failed

continued

Job	Clause	Type	Measured	Status
093 P6b	no-overlap 5-25% slower than base, gpt-oss 25%	band	0.2 [0.1, 0.2]	failed
093 P6c	no-overlap 5-25% slower than base, Qwen3 12.5%	band	-0.0 [-0.1, 0.0]	failed
093 P6d	no-overlap 5-25% slower than base, Qwen3 25%	band	0.1 [0.0, 0.2]	failed
093 P6e	all-CPU 0-10% slower than base, gpt-oss 11%	band	7.8 [7.2, 8.4]	held
093 P6f	all-CPU 0-10% slower than base, gpt-oss 25%	band	9.3 [8.7, 9.8]	held
093 P6g	all-CPU 0-10% slower than base, Qwen3 12.5%	band	14.7 [14.1, 15.3]	failed
093 P6h	all-CPU 0-10% slower than base, Qwen3 25%	band	14.6 [14.0, 15.2]	failed
093 P7a	oracle gain below 15% at the GPU-bound cell, gpt-oss 40%	threshold	48.5 [46.2, 50.9]	failed
093 P7b	oracle gain below 15% at the GPU-bound cell, Qwen3 43.75%	threshold	56.5 [53.3, 59.2]	failed
094 P1a	bypass oracle's reads within 15% of the optimum's, gpt-oss 11%	band	30.8	failed
094 P1b	bypass oracle's reads within 15% of the optimum's, gpt-oss 25%	band	31.1	failed
094 P1c	bypass oracle's reads within 15% of the optimum's, gpt-oss 40%	band	29.0	failed
094 P1d	bypass oracle's reads within 15% of the optimum's, Qwen3 12.5%	band	43.4	failed
094 P1e	bypass oracle's reads within 15% of the optimum's, Qwen3 25%	band	46.6	failed
094 P1f	bypass oracle's reads within 15% of the optimum's, Qwen3 43.75%	band	38.5	failed
094 P2a	bypass oracle faster than base, gpt-oss 11%	sign	1.033 [1.029, 1.038]	held
094 P2b	bypass oracle faster than base, gpt-oss 25%	sign	1.181 [1.175, 1.187]	held
094 P2c	bypass oracle faster than base, gpt-oss 40%	sign	1.232 [1.220, 1.243]	held
094 P2d	bypass oracle faster than base, Qwen3 12.5%	sign	1.002 [0.997, 1.006]	held (point)
094 P2e	bypass oracle faster than base, Qwen3 25%	sign	1.131 [1.122, 1.139]	held
094 P2f	bypass oracle faster than base, Qwen3 43.75%	sign	1.189 [1.178, 1.200]	held
094 P2g	bypass gain 10-45%, gpt-oss 11%	band	3.3 [2.9, 3.8]	failed
094 P2h	bypass gain 10-45%, gpt-oss 25%	band	18.1 [17.5, 18.7]	held
094 P2i	bypass gain 10-45%, Qwen3 12.5%	band	0.2 [-0.3, 0.6]	failed
094 P2j	bypass gain 10-45%, Qwen3 25%	band	13.1 [12.2, 13.9]	held
094 P3a	bypass hit rate within 4 points of the optimum's, gpt-oss 11%	band	-8.2	failed
094 P3b	bypass hit rate within 4 points of the optimum's, gpt-oss 25%	band	-3.2	held
094 P3c	bypass hit rate within 4 points of the optimum's, gpt-oss 40%	band	-1.4	held
094 P3d	bypass hit rate within 4 points of the optimum's, Qwen3 12.5%	band	-10.9	failed
094 P3e	bypass hit rate within 4 points of the optimum's, Qwen3 25%	band	-5.3	failed
094 P3f	bypass hit rate within 4 points of the optimum's, Qwen3 43.75%	band	-1.3	held
094 P4a	law over-predicts the bypass oracle by 10-40%, gpt-oss 11%	band	6.9	failed
094 P4b	law over-predicts the bypass oracle by 10-40%, gpt-oss 25%	band	5.8	failed
094 P4c	law over-predicts the bypass oracle by 10-40%, Qwen3 12.5%	band	4.7	failed
094 P4d	law over-predicts the bypass oracle by 10-40%, Qwen3 25%	band	8.5	failed
094 P5a	bypass beats prefetch by $\geq 15\%$, gpt-oss 11%	threshold	6.9	failed
094 P5b	bypass beats prefetch by $\geq 15\%$, Qwen3 12.5%	threshold	5.9	failed
094 P5c	bypass and prefetch within 15%, gpt-oss 25%	band	-27.6	failed
094 P5d	bypass and prefetch within 15%, gpt-oss 40%	band	-21.3	failed
094 P5e	bypass and prefetch within 15%, Qwen3 25%	band	-19.7	failed
094 P5f	bypass and prefetch within 15%, Qwen3 43.75%	band	-26.6	failed
094 P6a	W = 16 keeps $\geq 70\%$ of the W = all gain, gpt-oss 11%	threshold	176 [165, 190]	held
094 P6b	W = 16 keeps $\geq 70\%$ of the W = all gain, gpt-oss 25%	threshold	63	failed
094 P6c	W = 16 keeps $\geq 70\%$ of the W = all gain, gpt-oss 40%	threshold	34	failed
094 P6d	W = 16 keeps $\geq 70\%$ of the W = all gain, Qwen3 12.5%	threshold	–	<i>untested</i>
094 P6e	W = 16 keeps $\geq 70\%$ of the W = all gain, Qwen3 25%	threshold	94 [91, 96]	held

continued

Job	Clause	Type	Measured	Status
094 P6f	W = 16 keeps $\geq 70\%$ of the W = all gain, Qwen3 43.75%	threshold	53	failed
095 P1a	fetch oracle's reads within 10% of the simulation, gpt-oss 11%	band	-0.2	held
095 P1b	fetch oracle's reads within 10% of the simulation, gpt-oss 25%	band	-6.2	held
095 P1c	fetch oracle's reads within 10% of the simulation, gpt-oss 40%	band	-17.2	failed
095 P1d	fetch oracle's reads within 10% of the simulation, Qwen3 12.5%	band	-0.5	held
095 P1e	fetch oracle's reads within 10% of the simulation, Qwen3 25%	band	-4.4	held
095 P1f	fetch oracle's reads within 10% of the simulation, Qwen3 43.75%	band	-16.9	failed
095 P1g	fetch oracle's hit rate within 3 points of the simulation, gpt-oss 11%	band	0.1	held
095 P1h	fetch oracle's hit rate within 3 points of the simulation, gpt-oss 25%	band	0.8	held
095 P1i	fetch oracle's hit rate within 3 points of the simulation, gpt-oss 40%	band	1.1	held
095 P1j	fetch oracle's hit rate within 3 points of the simulation, Qwen3 12.5%	band	0.1	held
095 P1k	fetch oracle's hit rate within 3 points of the simulation, Qwen3 25%	band	0.5	held
095 P1l	fetch oracle's hit rate within 3 points of the simulation, Qwen3 43.75%	band	0.8	held
095 P2a	fetch oracle faster than base, gpt-oss 11%	sign	1.286 [1.279, 1.294]	held
095 P2b	fetch oracle faster than base, gpt-oss 25%	sign	1.316 [1.307, 1.326]	held
095 P2c	fetch oracle faster than base, gpt-oss 40%	sign	1.254 [1.242, 1.265]	held
095 P2d	fetch oracle faster than base, Qwen3 12.5%	sign	1.345 [1.338, 1.351]	held
095 P2e	fetch oracle faster than base, Qwen3 25%	sign	1.379 [1.368, 1.390]	held
095 P2f	fetch oracle faster than base, Qwen3 43.75%	sign	1.264 [1.251, 1.276]	held
095 P2g	law within 10% on the fetch oracle, gpt-oss 11%	band	-7.6 [-9.0, -6.3]	held
095 P2h	law within 10% on the fetch oracle, gpt-oss 25%	band	-13.4	failed
095 P2i	law within 10% on the fetch oracle, gpt-oss 40%	band	-13.6	failed
095 P2j	law within 10% on the fetch oracle, Qwen3 12.5%	band	-8.9 [-10.0, -7.8]	held
095 P2k	law within 10% on the fetch oracle, Qwen3 25%	band	-14.9	failed
095 P2l	law within 10% on the fetch oracle, Qwen3 43.75%	band	-16.9	failed
095 P3a	both2's reads within 25% of the simulation, gpt-oss 11%	band	-3.7	held
095 P3b	both2's reads within 25% of the simulation, gpt-oss 25%	band	-3.9	held
095 P3c	both2's reads within 25% of the simulation, gpt-oss 40%	band	-12.3	held
095 P3d	both2's reads within 25% of the simulation, Qwen3 12.5%	band	-1.9	held
095 P3e	both2's reads within 25% of the simulation, Qwen3 25%	band	-4.9	held
095 P3f	both2's reads within 25% of the simulation, Qwen3 43.75%	band	-8.7	held
095 P3g	both2's hit rate within 5 points of the simulation, gpt-oss 11%	band	-0.7	held
095 P3h	both2's hit rate within 5 points of the simulation, gpt-oss 25%	band	-1.6	held
095 P3i	both2's hit rate within 5 points of the simulation, gpt-oss 40%	band	-0.5	held
095 P3j	both2's hit rate within 5 points of the simulation, Qwen3 12.5%	band	0.2	held
095 P3k	both2's hit rate within 5 points of the simulation, Qwen3 25%	band	-0.4	held
095 P3l	both2's hit rate within 5 points of the simulation, Qwen3 43.75%	band	-0.3	held
095 P4a	both2 at 55-85% of the limit, gpt-oss 11%	band	63.9 [62.8, 65.0]	held
095 P4b	both2 at 55-85% of the limit, gpt-oss 25%	band	44.4	failed
095 P4c	both2 at 55-85% of the limit, Qwen3 12.5%	band	68.0 [67.1, 69.0]	held
095 P4d	both2 at 55-85% of the limit, Qwen3 25%	band	50.9	failed
095 P4e	both2 faster than the hit-optimal oracle, gpt-oss 11%	sign	1.532 [1.519, 1.546]	held

continued

Job	Clause	Type	Measured	Status
095 P4f	both2 faster than the hit-optimal oracle, gpt-oss 25%	sign	1.310 [1.291, 1.329]	held
095 P4g	both2 faster than the hit-optimal oracle, Qwen3 12.5%	sign	2.045 [2.016, 2.075]	held
095 P4h	both2 faster than the hit-optimal oracle, Qwen3 25%	sign	1.388 [1.374, 1.402]	held
095 P5a	law over-predicts both2 by 15-50%, gpt-oss 11%	band	12.7	failed
095 P5b	law over-predicts both2 by 15-50%, gpt-oss 25%	band	25.7 [23.2, 28.0]	held
095 P5c	law over-predicts both2 by 15-50%, Qwen3 12.5%	band	7.1	failed
095 P5d	law over-predicts both2 by 15-50%, Qwen3 25%	band	20.8 [17.8, 24.1]	held
095 P6a	both2 gains $\geq 25\%$ over base, gpt-oss 11%	threshold	39.4 [38.6, 40.3]	held
095 P6b	both2 gains $\geq 25\%$ over base, Qwen3 12.5%	threshold	42.1 [41.3, 42.9]	held
095 P7a	paced both3p within 10% of both2, gpt-oss 11%	band	4.2 [3.7, 4.7]	held
095 P7b	paced both3p within 10% of both2, gpt-oss 25%	band	11.8	failed
095 P7c	paced both3p within 10% of both2, gpt-oss 40%	band	7.3 [6.8, 7.8]	held
095 P7d	paced both3p within 10% of both2, Qwen3 12.5%	band	-0.5 [-0.8, -0.2]	held
095 P7e	paced both3p within 10% of both2, Qwen3 25%	band	13.0	failed
095 P7f	paced both3p within 10% of both2, Qwen3 43.75%	band	6.8 [6.1, 7.5]	held

C The law’s form and its failures

Is the max the right form? In 19 of the 29 configurations nothing is copied, so they test one roof. We therefore refit the law and its alternatives on the same 33 measurements under leave-one-host-out validation (each of the 7 hosts predicted with constants fitted on the other six): the token-level max of Eq. (1) with G refit has a median error of 3.1% (90th percentile 9.4%); the additive form $G + X_c/B_c + X_p/B_p$, 4.5% (11.3%); a per-layer max summed over layers from the engine’s per-layer miss histograms, 3.2% (10.2%). The additive form ties in sample but misses the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, by -11%, while the max is within $\pm 0.2\%$ there; on the slow-link copy rows the additive form is the closer of the two. The per-layer form neither beats the max nor explains its failures: the Jensen gap between a sum of per-layer maxima and the maximum of sums is a fifth of the Core Ultra’s residual, none of the EPYC 7352’s, and has the wrong sign on the EPYC 9655 and on the prefetch rows. We keep Eq. (1), with its evidence stated: the max beats the sum on the three fast-link copy rows and on the EPYC 9655 rows, where both fail, the sum is closer on the two slow-link copy rows, and elsewhere the two are within a few points, so the form is settled only where both paths carry most of the bytes. FreeToken’s own runs are not predicted, because its counters are not exposed; the law covers our engine and llama.cpp.

Where it fails, and why. Each failure now has a measured cause. (i) Copying misses on demand (FETCH) at 25% is over-predicted by +11.7% on the Core Ultra 7 and +11.4% on the EPYC 7352 (links of 36 and 26 GB/s); +8.8% and +8.5% with G refit. The engine’s per-layer timers show why: a CPU miss served beside a copy takes 20–62% longer than one served alone, so the two paths share DRAM less evenly than the probe’s combined rate says; a variant of the law that uses the probe’s per-path rates under contention and a per-layer hand-off constant predicts both rows within +3.1% and +4.1%, at a worse median (4.2%). (ii) llama.cpp on a Core Ultra 7 with 8 performance and 16 efficiency cores runs at 46% of the law at its default thread count, because it splits every matrix evenly across its threads; our helpers claim work dynamically and stay on the law on the same machine. We did not test llama.cpp restricted to the performance cores. (iii) On a 12-channel EPYC 9655 (559 GB/s) host memory stops being the limit (+16 to +29%): the per-layer timers show a floor of 55–70 μ s per layer with CPU misses that no desktop host has (their intercepts are at or below zero), the latency of the hand-off at a bandwidth where a 13 MB expert takes 24 μ s. (iv) Prefetch configurations are under-predicted by -8.7 to -3.4%: their copies overlap attention, which the law charges in full.

D The grid: other cards, and the scope of the gain

Table 7 lists every cell of Table 1’s protocol on the four machines we measured, each against its own limit, and Fig. 3 plots the fractions. The protocol was repeated on an RTX 4090 next to a Core i5-12400 (the host reads 37 GB/s, the link 25) and an RTX 3090 next to a Core i9-11900KF (45 GB/s; CPU and link together read *less* than the CPU alone, so the law’s table copies nothing), at the cells that fit 24 GB. On the 4090 ours runs $1.03\text{--}1.37\times$ FreeToken and $1.8\text{--}2.6\times$ llama.cpp at half host B’s absolute speeds, the host being the reason; on the 3090 it runs $1.6\text{--}2.2\times$ llama.cpp (FreeToken’s carried-over backend ran below llama.cpp there and is not counted). The cards’ own bandwidth never

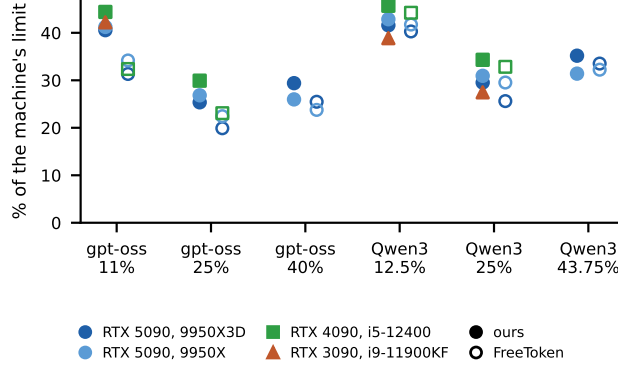


Figure 3: Ours (filled) and FreeToken (hollow) as a percentage of each machine’s own speed limit (exact optimum, the machine’s highest probed host rate, the card’s datasheet rate) at every cell of Table 7: two RTX 5090 hosts, an RTX 4090 and an RTX 3090 (FreeToken’s 3090 cells, its carried-over backend running below llama.cpp, are omitted). The limit ranges from 41 to 519 tok/s across these cells; ours stays at 25–46% of it.

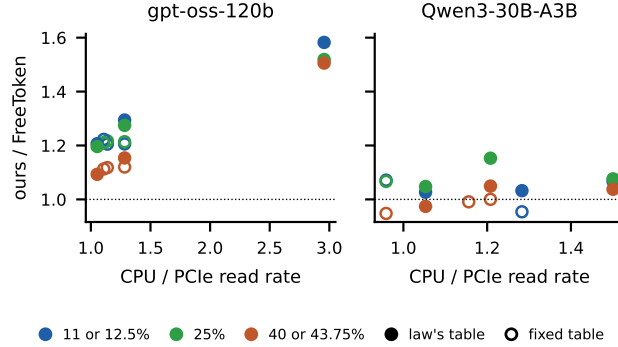


Figure 4: Ours over FreeToken’s better backend (carried over from host B on host S) in every same-machine comparison of the current engine on the two models of Table 1 (eight hosts), against the machine’s CPU-to-PCIe read ratio (FreeToken’s own probe). Filled: FETCH table from the law on that machine; hollow: the fixed table. The rightmost gpt-oss points are a Core i9-14900K with FreeToken’s executor on its performance cores; the rightmost Qwen3 points are a Ryzen 9 7900.

enters: every one of the seven cells on the 24 GB cards is host-bound at the limit, and ours stands at 30–46% and 27–42% of it. Mixtral-8x7B (8 experts per layer, top-2; 26 GB of experts on the 3090) marks the scope: with 2 of 8 experts resident the cache hits 31% of reads where whole-layer pinning hits 25%, so there is little locality to earn, and ours ties llama.cpp (1.015 [1.008, 1.024]); on the 128-expert models the same cache hits 53–79% at 11–25% resident. Across eight hosts the gpt-oss lead over FreeToken grows with the host’s CPU-to-link ratio, from 1.09–1.22 $\times$ where the CPU reads 1.05–1.14 $\times$ its link’s rate to 1.51–1.58 $\times$ behind a link of half the bandwidth, as the law predicts, and the Qwen3 lead stays at 0.95–1.15 $\times$ (Fig. 4, Appendix F).

The 24 GB cards run the cells that fit (the VRAM arithmetic, written on the machine from the GGUF header, skipped Qwen3 at 43.75%: 28.0 GB of slots, dense weights and KV against a 23 GB limit). The predictions for jobs 091 and 092 and their outcomes are in `prereg/grid_outcome_091.md` and `092.md`: on the 4090 our lead over FreeToken at gpt-oss 11% (37%) exceeded the predicted 10–35% band, ours ran 1.78 $\times$ llama.cpp at Qwen3 12.5% against a predicted 1.8, the law’s tables copied one more expert than host B’s at one entry each (the prediction reasoned from the link alone; the CPU is slower in the same proportion), and ours’ fraction of the limit at Qwen3 12.5% (45.7%) exceeded the predicted 25–45%; on the 3090 FreeToken’s carried-over backend ran below llama.cpp, ours ran 1.56–1.64 $\times$ llama.cpp at two cells against a predicted 1.8, Mixtral tied at $C=2$ against a predicted 1.3 $\times$ and its $C=4$ cell was lost to the job’s deadline, and one launch-order check exceeded 3% (7%).

Table 7: The grid: Table 1’s protocol on every card and host we measured, each cell against its own machine’s limit (the exact optimum’s reads on the same trace, the machine’s highest probed host rate, the card’s datasheet rate). *Bound*: what binds the limit; *host* when the optimum’s reads alone take longer than the GPU’s whole read, so the GPU has slack, *both* when the limit runs experts on the CPU until the two paths take equally long, which is where the all-in-VRAM speed can be exceeded. Ours and FreeToken are launch 2 (FreeToken’s backend carried over from host B’s selection); the 40 and 43.75% budgets do not fit a 24 GB card. [†]On the RTX 3090 FreeToken’s carried-over hybrid backend ran below llama.cpp; its own calibration recommends its offload backend there, which we did not run, so those cells are reported, not counted. Mixtral-8x7B (26 GB of experts, top-2 of 8) is ours against llama.cpp only, on the RTX 3090, launch 1.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Limit (tok/s)	Bound	Ours, % of limit
RTX 5090, 9950X3D (host B): B_{host} 88 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,1,2 / 0,0,1,1,2,3,3,4,5									
gpt-oss-120b	11%	34.9	54.0	69.9	1.29 [1.28, 1.31]	2.00	172	host	41
gpt-oss-120b	25%	40.0	85.6	109.2	1.27 [1.25, 1.29]	2.73	430	host	25
gpt-oss-120b	40%	47.7	132.2	152.5	1.15 [1.13, 1.17]	3.20	519	both	29
Qwen3-30B-A3B	12.5%	19.5	38.7	40.0	1.03 [1.02, 1.04]	2.05	96	host	42
Qwen3-30B-A3B	25%	22.1	54.7	63.1	1.15 [1.13, 1.17]	2.86	214	host	30
Qwen3-30B-A3B	43.75%	28.4	103.1	108.2	1.05 [1.04, 1.06]	3.81	308	both	35
RTX 5090, 9950X (host S): B_{host} 71 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,2,3 / 0,0,1,2,2,3,4,5,6									
gpt-oss-120b	11%	28.9	48.0	57.9	1.21 [1.19, 1.22]	2.00	140	host	41
gpt-oss-120b	25%	33.6	78.7	94.1	1.20 [1.16, 1.22]	2.80	351	host	27
gpt-oss-120b	40%	40.1	122.2	133.6	1.09 [1.08, 1.11]	3.34	514	both	26
Qwen3-30B-A3B	12.5%	16.1	32.7	33.5	1.03 [1.02, 1.04]	2.08	78	host	43
Qwen3-30B-A3B	25%	18.5	51.4	53.9	1.05 [1.04, 1.06]	2.91	174	host	31
Qwen3-30B-A3B	43.75%	23.8	98.3	95.8	0.97 [0.96, 0.99]	4.02	305	both	31
RTX 4090, i5-12400 (job 091): B_{host} 37 GB/s, B_{gpu} 1008 GB/s; tables 0,0,1,2,2 / 0,0,1,1,2,3,4,4,5									
gpt-oss-120b	11%	18.1	23.8	32.7	1.37 [1.35, 1.39]	1.81	74	host	44
gpt-oss-120b	25%	21.0	42.5	55.0	1.30 [1.26, 1.33]	2.62	184	host	30
Qwen3-30B-A3B	12.5%	10.5	18.2	18.8	1.03 [1.02, 1.05]	1.78	41	host	46
Qwen3-30B-A3B	25%	12.3	30.0	31.3	1.05 [1.03, 1.06]	2.54	91	host	34
RTX 3090, i9-11900KF (job 092): B_{host} 45 GB/s, B_{gpu} 936 GB/s; tables 0,0,0,0,0 / 0,0,0,0,0,0,0,0									
gpt-oss-120b	11%	22.8	8.4 [†]	37.3	4.46 [4.25, 4.69] [†]	1.64	88	host	42
Qwen3-30B-A3B	12.5%	12.3	9.0 [†]	19.2	2.12 [2.02, 2.21] [†]	1.56	49	host	39
Qwen3-30B-A3B	25%	13.8	11.2 [†]	30.1	2.68 [2.53, 2.83] [†]	2.18	110	host	27
Mixtral-8x7B Q4_K_M	$C=2$ (25%)	8.22	—	8.34	—	1.015 [1.008, 1.024]	—	—	—

E Foresight in the engine

Three jobs ran oracle policies in the engine, each on a rented RTX 5090 host with weaker host memory than host B’s. Table 8 lists every cell of the single-read factorial of Section 5 on host O3 (a Ryzen 9 9950X whose probe reads 53 GB/s by the CPU at the helper count, 43 over the link, 60 with both and 61.5 at best; the law’s FETCH tables 0,0,1,2,2 for gpt-oss and 0,0,1,1,2,3,4,4,5 for Qwen3), and Table 9 the milliseconds per token of the engine states the three jobs realised beside each host’s limit and the model’s values. Table 10 lists the first job (093), the hit-optimal oracle with a window sweep on host O1 (a Ryzen 9 9950X: 44, 46, 49 and 50.5 GB/s; tables 0,0,1,2,3 and 0,0,1,2,3,4,5,6,7), and Table 11 the second (094), MIN with bypass with background admissions beside the hit-optimal oracle on host O2 (a Ryzen 9 7950X: 44, 46, 50 and 50.6 GB/s; the same tables). On O1 and O2 every cell but gpt-oss 40% and Qwen3 43.75% is host-bound at the limit; on O3, whose memory is stronger, the same four.

The single-read oracles. In the fetch oracle the step’s plan is computed before the layer runs from the trace’s record of the current step: the misses MIN with bypass would admit are fetched by the engine’s FETCH operation into their victims’ slots and run from there, and every other miss runs on the CPU; the oracle refuses an admission when no resident’s next use is further than the candidate’s. The single-read prefetch admits, each step, the non-resident experts whose first use is at least d steps away ($d=2$ unpaced, 3 paced), provided the victim’s next use is further than the candidate’s use after that first one; an expert used sooner than d steps away misses on the CPU, or is fetched when the two are combined. The same policies simulated on the trace with the engine’s semantics (the lookahead within the sequence, residents carried over) predict the engine’s reads and hit rates: the fetch oracle’s hit rate within 1.1 points and its reads within -17 to 0% (the engine reads fewer at the GPU-bound cells, where its cache is warm

Table 8: The single-read oracles (job 095; RTX 5090, Ryzen 9 9950X, host memory 61.5 GB/s at best) beside the hit-optimal one. Per cell: the online policy’s speed and its share of this host’s limit; for each oracle with the whole sequence in view, its ratio to the online policy (paired, 95% interval), hit rate, host reads per token (CPU misses + fetches + admissions) as a multiple of the optimum’s (each read counted once), share of the limit and the law on its own counters against its measured time. *fetch*: MIN with bypass, admitted misses fetched into their slot this step; *lead*: the scheduled single-read prefetch (first use at least two steps away); *both*: fetch and lead; *paced*: both on the paced copy path with a three-step lead; *hit-opt.*: the hit-optimal oracle (Belady prefetch without bypass, unpaced). The simulation columns give the trace’s prediction of reads and hits for fetch and both.

Cell	online			fetch			law	both			hit-opt.	lead			paced		simulated					
	tok/s	% lim.	ratio	hits	reads/R*	% lim.		ratio	hits	reads/R*	% lim.	ratio	hits	reads/R*	% lim.	ratio	% lim.	ratio	% lim.	fetch reads	hits	both reads
gpt-oss 11%	55.5	46	1.29 [1.28, 1.29]	72	1.04	59 -8%	1.39 [1.39, 1.40]	86	1.23	64 0.91 [0.90, 0.92]	98 1.77	42 1.26	58 1.45	67					40	72	49	86
gpt-oss 25%	90.3	30	1.32 [1.31, 1.33]	89	1.08	39 -13%	1.49 [1.48, 1.50]	97	1.45	44 1.14 [1.12, 1.15]	99 2.11	34 1.45	43 1.66	50					18	88	23	99
gpt-oss 40%	128.8	25	1.25 [1.24, 1.27]	95	1.14	32 -14%	1.39 [1.37, 1.41]	99	1.60	35 1.27 [1.25, 1.28]	99 2.20	32 1.38	35 1.49	38					9	94	12	100
Qwen3 12.5%	32.3	48	1.34 [1.34, 1.35]	74	1.04	64 -9%	1.42 [1.41, 1.43]	84	1.20	68 0.69 [0.68, 0.71]	77 2.47	33 1.22	58 1.41	68					101	74	118	84
Qwen3 25%	51.8	34	1.38 [1.37, 1.39]	88	1.04	48 -15%	1.48 [1.46, 1.49]	96	1.42	51 1.06 [1.05, 1.08]	99 2.00	37 1.41	48 1.67	57					47	88	65	97
Qwen3 43.75%	91.4	30	1.26 [1.25, 1.28]	96	1.08	38 -17%	1.41 [1.39, 1.42]	99	1.62	42 1.27 [1.26, 1.29]	100 2.29	38 1.40	42 1.50	45					18	95	24	99

Table 9: The accounting, measured: milliseconds per token of the engine’s states that the oracle jobs realised, on each oracle host, beside that host’s limit and the model’s values for the states it prices. *online*: the deployed policy; *no ovl.*: its within-layer overlap off; *all CPU*: no FETCH table; *bypass*: MIN with bypass with background admissions (two reads per admitted expert); *hit-opt.*: Belady prefetch without bypass, which admits what the optimum bypasses (job 095’s unpaced); *fetch*: MIN with bypass, admitted misses fetched into their slot (one read, serialised: the model’s foresight-only state $v(F)$); *both*: fetch and the scheduled single-read prefetch (one read, overlapped: the model’s $v(O, F)$); *paced*: both on the paced copy path with a three-step lead; *limit*: the limit on that host; $v(F)$, $v(O, F)$, $v(\text{all})$: the model.

Host	Cell	online	no ovl.	all CPU	bypass	hit-opt.	fetch	both	paced	limit	$v(F)$	$v(O, F)$	$v(\text{all})$
O1 (9950X, job 093)	gpt-oss 11%	21.12	21.01	22.91	—	22.88	—	—	—	10.06	14.54	11.27	10.06
O1 (9950X, job 093)	gpt-oss 25%	12.89	12.91	14.21	—	9.22	—	—	—	4.03	8.04	5.10	4.03
O1 (9950X, job 093)	gpt-oss 40%	8.90	8.95	—	—	5.99	—	—	—	1.97	5.75	3.84	1.97
O1 (9950X, job 093)	Qwen3 12.5%	36.92	36.91	43.29	—	40.29	—	—	—	18.04	24.90	19.29	18.04
O1 (9950X, job 093)	Qwen3 25%	22.73	22.76	26.60	—	18.22	—	—	—	8.11	14.04	9.22	8.11
O1 (9950X, job 093)	Qwen3 43.75%	12.71	12.67	—	—	8.12	—	—	—	3.32	8.25	5.51	3.32
O2 (7950X, job 094)	gpt-oss 11%	20.85	—	—	20.17	21.57	—	—	—	10.04	14.27	11.96	10.04
O2 (7950X, job 094)	gpt-oss 25%	12.72	—	—	10.77	7.80	—	—	—	4.02	7.86	5.86	4.02
O2 (7950X, job 094)	gpt-oss 40%	8.79	—	—	7.13	5.62	—	—	—	1.97	5.63	3.78	1.97
O2 (7950X, job 094)	Qwen3 12.5%	36.64	—	—	36.58	38.74	—	—	—	18.00	24.70	20.06	18.00
O2 (7950X, job 094)	Qwen3 25%	22.24	—	—	19.67	15.80	—	—	—	8.09	13.58	9.90	8.09
O2 (7950X, job 094)	Qwen3 43.75%	12.02	—	—	10.11	7.42	—	—	—	3.32	7.56	4.90	3.32
O3 (9950X, job 095)	gpt-oss 11%	18.01	—	—	—	19.79	14.00	12.91	12.39	8.26	12.63	9.49	8.26
O3 (9950X, job 095)	gpt-oss 25%	11.08	—	—	—	9.74	8.42	7.44	6.66	3.31	7.05	4.28	3.31
O3 (9950X, job 095)	gpt-oss 40%	7.77	—	—	—	6.14	6.20	5.58	5.20	1.95	5.24	3.56	1.95
O3 (9950X, job 095)	Qwen3 12.5%	30.95	—	—	—	44.54	23.02	21.78	21.89	14.81	20.35	16.06	14.81
O3 (9950X, job 095)	Qwen3 25%	19.32	—	—	—	18.17	14.01	13.09	11.58	6.66	11.68	7.76	6.66
O3 (9950X, job 095)	Qwen3 43.75%	10.94	—	—	—	8.59	8.66	7.78	7.28	3.30	7.24	4.76	3.30

from the previous configuration’s run and the simulation starts cold), the combined oracle’s hits within two points (Table 8). The paced copy path copies one 16 MB piece at a time on a single copier thread and publishes a landed copy at the start of the next step; the unpaced path copies each admission as one transfer. The hit-optimal oracle’s background admissions drive host memory at 71–82% of the probe’s best rate at the host-bound cells, the prefetching single-read oracles at 64–85% (the paced one 71–82%), the online policy itself at 66–82%, the fetch oracle at 42–67%. The predictions and their outcomes are in `prereg/foresight.outcome.095.md`: of 56 clauses, 44 held and 12 failed, all of size (Appendix B).

The double-reading oracles. Both read the routing of the next W decode steps; their copies ride the paced admission path, and an expert still in flight at its step runs on the CPU like any other miss. The *hit-optimal* oracle admits the experts those steps will route to, soonest first, evicting the resident whose next use is furthest (Belady within the window), for one future use as readily as for ten. The *bytes-optimal* oracle is MIN with bypass with its admissions made through the background path: a miss is served where it is, by the CPU or a demand copy, and admitted only if its next use comes before the furthest next use among the residents. With the whole sequence in view the hit-optimal oracle runs $1.40 \times [1.38, 1.41]$ the online policy at gpt-oss 25% and $1.25 \times [1.24, 1.26]$ at Qwen3 25% on host O1, $1.63 \times$ and $1.41 \times$ on O2, and 1.48 – $1.62 \times$ at the two GPU-bound cells; half of its gain at gpt-oss 25% arrives between $W=4$ and 16 (9.2 tokens by log interpolation; Qwen3 25%: 6.1 against the trace’s 3.9). Where the host binds hardest it loses, 3–8% at gpt-oss 11% and Qwen3 12.5% on both hosts, and at every window on O1 (0.78 – $0.92 \times$): it reads more host bytes than the online policy (82 and 195 experts per token against 63 and 160), 2.02 – $2.18 \times$ the

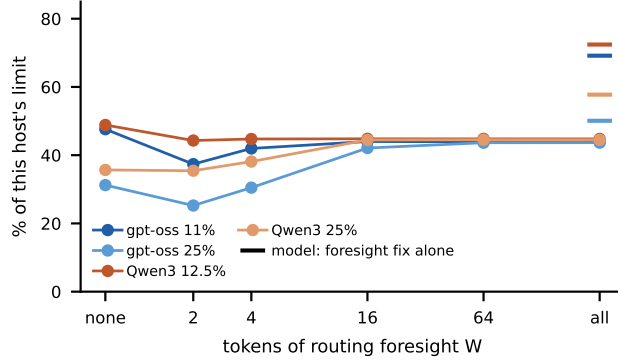


Figure 5: The first oracle job (093, host O1): speed as a percentage of this host’s limit against the window W of future routing the hit-optimal oracle sees, at the four host-bound cells. “none” is the online policy; “all” the whole sequence; points are connected for the trend only. The tick at the right is the model’s foresight-only term for the cell, the speed it predicts with that one fix; the oracle, reading every admitted expert twice, stays at 44–45% of the limit at every window.

optimum’s R^* , because Belady admits an expert used once where the optimum bypasses it and re-copies what it evicts (its admissions, each one read, are most of its reads; a miss served on the CPU before its copy lands is the second read of that expert). An unpaced copy path, run at the four host-bound cells of O1, reaches 98–99% hits at three of them and still loses at gpt-oss 11% and Qwen3 12.5% (where it hits 75%), so copies that arrive in time do not remove the loss there: the extra bytes cause it. The bytes-optimal oracle gains 13–23% at the four cells of 25% and above and 0–3% at the two below (6% with a 16-token window); its misses fall (30 to 20 and 85 to 64 per token at 25%) but its total reads, misses plus the admissions that read each admitted expert again, are $1.73\text{--}2.04 \times R^*$ and exceed the online policy’s by 4–5% at the two lowest budgets; the law on its own counters predicts its time within 9% at every cell, and the bytes it saves account for 4–21 points of its 13–23% gain at the four cells (most of it at gpt-oss 40% and Qwen3 43.75%, a third at Qwen3 25%). The hit-optimal oracle reads $1.01\text{--}1.13 \times$ the bytes-optimal one’s host bytes on O2 and gains 41–63% at the same four cells, and the law, with its GPU term calibrated on the online run, over-predicts those runs by 37–53% (on O1, 24–39%): its gain is the overlap of copies issued ahead of their use with the step, which the law charges in sequence. Pacing the online policy’s own copies changes it by under 1% ($0.997\text{--}1.005 \times$). With no FETCH table (every miss on the CPU) the online policy runs 8–15% slower at the four host-bound cells; with the engine’s within-layer overlap of a layer’s CPU misses with its GPU hits turned off it changes by under 1% at every cell (0.5% faster to 0.6% slower), so the overlap the accounting charges is across the step and in rebalancing, not within a layer. The bytes-optimal oracle with a 16-token window keeps 34–176% of the whole-sequence gain at the five cells where that gain’s interval excludes zero (more than all of it at gpt-oss 11%, where the shorter window admits less).

Why the bytes-optimal oracle’s misses stay above the optimum’s reads. Besides the second read, its misses alone are $1.31\text{--}1.47 \times$ the optimum’s reads at the four host-bound cells (at the two GPU-bound ones the ideal simulation below is itself $1.3\text{--}1.4 \times$ the optimum, which sees across sequences, and the engine’s misses are below it at gpt-oss 40% and just above it at Qwen3 43.75%). The engine’s paced copy path moves one 16 MB piece at a time from pageable host memory on a single copier thread, behind the demand copies, publishes a landed copy at the start of the next step, and admits nothing while 64 copies are queued; at the 16–30 admissions per token (210–285 MB) of the two lowest budgets it lags the admissions by steps, and an expert still in flight is a miss the CPU reads again. The last four columns of Table 11 simulate MIN with bypass on the measured routing trace with the engine’s own semantics (the lookahead within the sequence, residents carried over, no queue cap) and a copy that lands d steps after it is issued, during which the expert misses and is not re-admitted: at $d=1$, the ideal, the misses come within 2–14% of the limit’s optimum at the host-bound cells; the engine’s misses meet the simulation at $d=2.6\text{--}3.6$ there, about three steps, and at or below 1.4 at the GPU-bound cells. At the fitted latency the simulation admits $1.3\text{--}2.0 \times$ what the engine admitted, so the queue cap, not only the latency, keeps the engine’s misses above the ideal. The predictions and their outcomes are in `prereg/foresight_outcome_093.md` and `094.md`: of 36 clauses for the first job, 9 held, 2 held on the point estimate, 21 failed and 4 were untested; of 38 for the second, 12 held, 1 on the point estimate, 24 failed and 1 was untested (Appendix B). The second job’s failures are of size, not direction: the bypass

Table 10: The hit-optimal oracle in the engine (job 093; RTX 5090, Ryzen 9 9950X, host memory 50.5 GB/s at best). tok/s of the online policy and of the oracle with the whole sequence in view; their ratio, paired by sequence, with its 95% interval; hit rate and misses per token of each (experts read from host memory by the CPU or by a demand copy), the optimum’s reads in parentheses, and the oracle’s background admissions (Belady prefetch without bypass: it admits what the optimum bypasses); speed as a percentage of this host’s limit; the accounting’s foresight-only term $v(F)$ for the cell in tok/s; the share of that term the oracle recovered and the share of the gap to the limit it closed.

Cell	online	oracle	ratio	hits % (opt.)	misses/token (opt.)	oracle adm.	% of limit	$v(F)$	recovered %	closed %
gpt-oss 11%	47.4	43.7	0.92 [0.91, 0.93]	58 → 67 (73)	60 → 47 (38)	35	47.6 → 44.0	68.8	−27	−16
gpt-oss 25%	77.6	108.5	1.40 [1.38, 1.41]	79 → 92 (89)	30 → 11 (15)	20	31.2 → 43.7	124.4	76	41
gpt-oss 40%	112.4	166.9	1.48 [1.46, 1.51]	89 → 98 (95)	15 → 3 (7)	12	22.1 → 32.8	174.0	92	42
Qwen3 12.5%	27.1	24.8	0.92 [0.91, 0.92]	60 → 66 (75)	155 → 132 (97)	63	48.9 → 44.8	40.2	−28	−18
Qwen3 25%	44.0	54.9	1.25 [1.24, 1.26]	78 → 89 (89)	85 → 42 (43)	46	35.7 → 44.5	71.2	52	31
Qwen3 43.75%	78.7	123.1	1.56 [1.53, 1.59]	92 → 99 (96)	32 → 5 (14)	25	26.1 → 40.8	121.3	103	49

Table 11: The bytes-optimal oracle (MIN with bypass) beside the hit-optimal one on the second oracle host (RTX 5090, Ryzen 9 7950X, host memory 50.6 GB/s at best). Per cell: the online policy’s speed; each oracle’s ratio to it with the whole sequence in view (paired, 95% interval), its hit rate, its misses per token (experts read from host memory by the CPU or by a demand copy; its background admissions in parentheses: the bypass oracle’s are this step’s CPU-served misses, each read a second time by the copy, the hit-optimal one’s are prefetches of what the optimum would bypass as readily as of what it would admit) and the law on its own counters against its measured time; the optimum’s reads and hit rate from the trace; and MIN with bypass simulated on the trace with the engine’s lookahead (within the sequence) under a copy latency of 1 (ideal), 2 and 3 steps, with the latency at which the simulation meets the engine’s misses.

Cell	online	ratio	bypass oracle, W=all				hit-optimal oracle, W=all				optimum		simulated misses/token				
			hits	misses	(adm.)	law	ratio	hits	misses	(adm.)	law	reads	hits	d=1	2	3	fit
gpt-oss 11%	48.0	1.03 [1.03, 1.04]	65	50 (16)	+7%	0.97 [0.96, 0.98]	71	42 (39)	+19%	38.3	73	39	44	48			3.6
gpt-oss 25%	78.6	1.18 [1.18, 1.19]	86	20 (9)	+6%	1.63 [1.61, 1.66]	95	7 (23)	+53%	15.3	89	18	19	21			2.6
gpt-oss 40%	113.8	1.23 [1.22, 1.24]	94	9 (4)	+2%	1.56 [1.54, 1.59]	98	3 (12)	+39%	6.8	95	9	10	11	below ideal		
Qwen3 12.5%	27.3	1.00 [1.00, 1.01]	64	138 (30)	+5%	0.95 [0.94, 0.95]	67	128 (65)	+11%	96.5	75	98	124	145			2.7
Qwen3 25%	45.0	1.13 [1.12, 1.14]	83	64 (21)	+9%	1.41 [1.39, 1.43]	91	34 (52)	+37%	43.4	89	47	57	65			2.8
Qwen3 43.75%	83.2	1.19 [1.18, 1.20]	95	19 (9)	+1%	1.62 [1.58, 1.65]	99	3 (27)	+47%	13.7	96	18	21	24			1.4

oracle’s misses were predicted within 15% of the optimum’s reads and came 29–47% above (the copy latency and the queue cap); its hit rate was predicted within 4 points of the optimum’s and fell 5–11 points short at three cells; its gain was predicted at 10–45% at the host-bound cells and came below that at the two lowest budgets; the law was predicted to over-predict it by 10–40% at the host-bound cells and over-predicted by 5–9%; and the two oracles were predicted within 15% of each other at 25% and above, where the hit-optimal one was 1.24–1.38× faster.

F The expert cache in more detail

This section holds the measurements of Section 2 that the main text summarises: where the cache’s gain comes from, the choice of the FETCH split, the hosts with half the link, long outputs, FreeToken’s own headline model, two other llama.cpp-based systems, output parity, and the cost of prompts and batches. Job numbers refer to the scripts on the gpu branch.

Where the gain comes from. Figure 6 builds the system up from stock llama.cpp on one machine. Placing each layer’s hot experts is the first gain: a static cache of the 32 experts per layer most requested on other text (the models’ own MATH-500 outputs) runs 1.31× and 1.89× stock, which pins whole layers instead. Its hit rate (52% and 67%) depends on how well the profile matches the workload: profiled in hindsight on the measured problems it hits 75% and 72% and runs faster (hatched). Replacing experts online closes that gap without a profile, if admissions are rationed: LRU admits every miss, and on Qwen3 its copies raise host reads from 127 to 188 experts per token, so it runs 0.88× the static cache (1.30× on gpt-oss); the decayed-frequency policy reads 41 and 118 per token, runs 1.58× and 1.12× the profiled static cache, and the hindsight cache runs 0.98 [0.95, 1.01] and 1.01 [0.98, 1.04] times as fast as it. On traces LRU misses only 2–11% more than decayed frequency at these budgets (Section 6); the rest of its cost here is the copies it admits. On host S, with everything else as in the full system, LRU runs 6–21% slower than decayed frequency across the six cells of Table 1 and trails FreeToken at four of them: the admission policy is most of the lead over FreeToken (77–84% of it at gpt-oss 25 and 40%, more than the whole of it at gpt-oss 11% and Qwen3 12.5 and

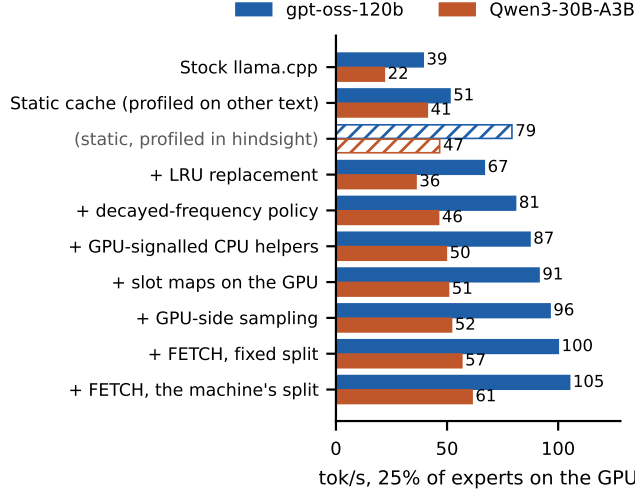


Figure 6: Ablation at 25% of experts on the GPU (15 AIME-25 problems, 256 tokens, host A). Each step adds one component to the one above; the hatched row is not a step: a static cache of each layer’s 32 most requested experts counted on the measured problems themselves.

Table 12: FETCH tables at 43.75% on Qwen3-30B-A3B (launch 1): entry n is how many of a layer’s n missed experts are copied over PCIe; the rest run on the CPU. Ratio to the fixed table, paired, 95% CI.

Table	$f(0..8)$	tok/s	vs fixed
Lite	0,0,1,1,1,2,2,3,3	109.0	1.057 [1.052, 1.062]
Law	0,0,1,1,2,3,3,4,5	108.0	1.048 [1.041, 1.055]
Law, fetch a single miss	0,1,1,1,2,3,3,4,5	104.1	1.010 [1.007, 1.012]
Fixed	0,1,1,2,3,3,4,5,6	103.1	1
None (CPU only)	0,...,0	100.7	0.977 [0.970, 0.984]
All (fetch every miss)	0,1,...,8	98.7	0.957 [0.953, 0.962]

25%). The GPU-signalled CPU helpers, GPU slot maps, GPU sampling and FETCH add 2–9% each, and the law’s FETCH table 5–8% over the fixed one. The full system runs $2.68\times$ (gpt-oss) and $2.81\times$ (Qwen3) stock llama.cpp.

Choosing the split per machine. On host B, the law’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0% (launch 1). Table 12 shows why at one budget: a single missed expert should run on the CPU (about $155\mu\text{s}$, hidden behind the GPU’s other experts) rather than be copied (about $225\mu\text{s}$, on the critical path); that entry alone is worth 3.7%. Fetching everything, FreeToken’s offload strategy expressed in our engine, is the worst table, and running everything on the CPU the second worst. The law’s table was within 1% of the best of five alternatives on launch 1 and tied with it on launch 2. Given each run’s measured misses, Eq. (2) ranks the tables correctly except law against lite, which are within 1%, and overstates their differences about $1.8\times$.

Machines with half the link. On RTX 5090s whose links we measured at 27–30 GB/s instead of 53 (jobs 085, 085b and 088: gpt-oss next to a Core i9-14900K, Qwen3 next to a Ryzen 9 7900 in a PCIe 4.0 slot), the law’s tables fetch less (gpt-oss 0,0,0,1,1, Qwen3 0,0,1,1,2,2,3,3,4) and beat the fixed table by 21–34% on gpt-oss and 9–18% on Qwen3, about twice the gain on host B for Qwen3 and four to six times for gpt-oss: the per-machine split matters most where the link is slow. On the Ryzen, ours leads FreeToken by $1.04\text{--}1.08\times$; with the fixed table it would trail at 12.5%. On the Core i9 the first comparison (job 085) was confounded: FreeToken’s hybrid backend ran gpt-oss at 12–66 tok/s, below its own offload backend (25–86), because its CPU executor spread 23 threads across the 8

performance and 16 efficiency cores and its calibration then sent two thirds of the misses over the slow link, the same failure Section 3 reports for llama.cpp; the 1.4–2.0 $\times$ lead that gave us measured a thread configuration. We reran it (job 088, a second rental of the same CPU model) with FreeToken’s executor on 8 threads pinned to the performance cores: its hybrid backend then runs 1.4–3.1 $\times$ faster than before (37.5, 62.1 and 92.5 tok/s; its calibration now copies 51.7% of misses), beating its own offload backend at 11 and 25% and tying it at 40%, and ours leads its better configuration by 1.58, 1.52 and 1.51 $\times$ (lowest interval bound 1.47). Restricted to the CPU alone, FreeToken’s 8-thread executor decodes 13.7 tok/s at every budget. llama.cpp on the same rental runs 40.0 tok/s with one thread per performance core (-9.4% from the law’s prediction) and 32.8 with all 24 threads (+25.9%): the hybrid-core failure is a thread-count choice, and the law holds once the choice is right. The law predicted llama.cpp on the Ryzen within +2.7%. Figure 4 puts every same-machine comparison on one axis.

Long outputs. Our lead does not come from the first 256 tokens. On gpt-oss, on ten held-out hard problems (AIME 2022, problems 11–15 of both exams) that neither system has finished reasoning about after 2,048 tokens (at most 1 of 10 reach an answer), ours runs 1.36 [1.34, 1.38], 1.32 [1.30, 1.34] and 1.19 [1.17, 1.21] times FreeToken at 11, 25 and 40% over all 2,048 tokens, and 1.37 [1.34, 1.39], 1.33 [1.31, 1.35] and 1.19 [1.17, 1.22] over tokens 257–2,048 (job 083, host A); both engines stream one event per token. On Qwen3, over tokens 257–2,048 of ten held-out MATH-500 problems, ours runs 1.043 [1.020, 1.070], 1.056 [1.036, 1.077] and 1.054 [1.025, 1.086] times FreeToken at 12.5, 25 and 43.75%. Both leads are larger than over the same runs’ first 256 tokens. Turning off FreeToken’s prefill overlap, which lends it cache slots during each prompt, changes its overall speed by 1% or less.

FreeToken’s own headline model. On FreeToken’s own headline model, Qwen3.6-35B-A3B in BF16 (256 experts of 6.3 MB per layer, hybrid linear attention), the two systems are close: ours runs 1.030 [1.023, 1.038] times FreeToken at 12.5% and ties at 25 and 37.5% (1.010 [0.998, 1.021], 1.003 [0.994, 1.013]), at 1.98–2.48 $\times$ llama.cpp (job 086, host A). FreeToken as shipped, with its automatic backend and cache size, runs this model at 93.8 tok/s on that machine, above the 77–83 tok/s its paper reports on an RTX 5090, so our setup does not handicap it; FreeToken’s fetch cap and thread count were left at their defaults, while our FETCH table was swept. Our lead is therefore model-dependent: largest with few large experts (gpt-oss, top-4 of 128), smallest with many small ones.

Other systems and outputs. Two newer llama.cpp-based systems, Pipelined Sharding [38] and leloch’s expert cache [21], run gpt-oss-120b at 1.28–1.44 $\times$ stock llama.cpp; ours ran 1.42–2.34 $\times$ the better of them (job 077). Teacher-forced on fixed text (1536 steps per model) against stock llama.cpp with every weight in the VRAM of an RTX PRO 6000 (job 090), our cache at 25% has a mean KL divergence over the full vocabulary of 0.0019 nats on gpt-oss and 0.0005 on Qwen3 (99.9th percentile 0.121 and 0.059, maximum 0.21 and 0.13), picks the same next token on 98.5% and 99.3% of steps, and changes the loss by +0.16% and +0.19%. Stock llama.cpp’s own `--n-cpu-moe` placement diverges from the same reference by as much (mean KL 0.0017 and 0.0008, agreement 98.8% and 99.3%): the differences are those of running experts on the CPU (the helpers quantise activations to Q8_0 where the GPU path uses Q8_1), not of the cache. Task accuracy was not measured.

Prompts and batches. The cache serves single-token steps; prompts and batches above one run llama.cpp’s stock graph with every expert in host memory, and pay for the cache’s resident experts and its one-thread CPU side. In llama-batched-bench at 25% on host S (job 089, both models) our prefill at 512 and 2,048 tokens runs 79–83% of stock llama.cpp’s and our decode at 2 and 4 parallel sequences 77–79% of it (the tool feeds both engines the same random tokens).

G The audit’s rows

Table 13 lists the 52 published measurements of Section 4 with their bounds. The sources are Fiddler [19], ProMoE [35], KTransformers [5], HybriMoE [53], Fate [9], SP-MoE [6], MoE-SpeQ [39], the CPU–GPU collaborative inference of Huang et al. [15], FreeToken [46], SeqMoE [41], NVIDIA’s pipelined sharding [38], a llama.cpp fork [21] and a llama.cpp pull request [7]. Three caveats apply to every row: the ceilings are datasheet ones where ours are probed; the trace is the model’s own sampled text, sometimes of a sibling variant (an FP8 trace for a Q4 deployment); and systems whose cache is pooled across layers, whose placement pins whole layers, or which verify several tokens per pass are scored against the per-layer one-token bound, which is not theirs.

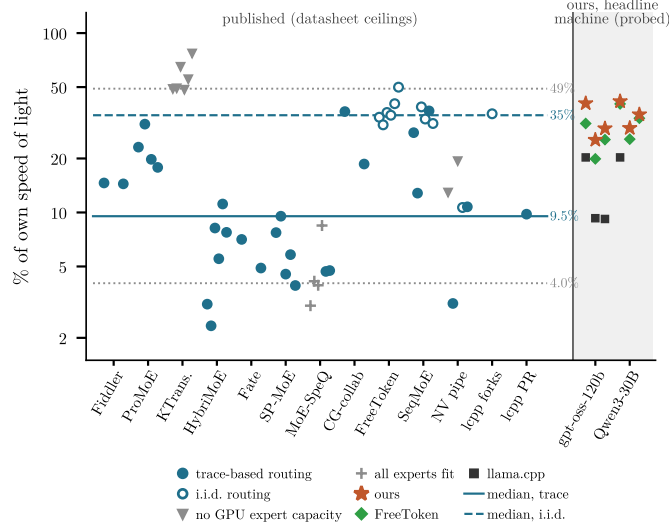


Figure 7: Published MoE-offloading measurements (52 rows from 13 sources) as a percentage of their own speed of light, computed from the paper’s hardware at datasheet rates and, where a routing trace of the model exists, the per-layer Belady optimum (filled); hollow points assume uniform routing, which overstates the optimum’s reads and so the percentage; the dotted lines are the class medians (no GPU expert capacity, 49.0%; every expert resident, 4.0%). Right: our cache, FreeToken and stock llama.cpp on host B against the probed limit of Table 1.

Table 13: The audit’s rows (Section 4): every published batch-1 measurement we scored, with the system’s stated hardware, its experts per layer resident on the GPU (C), the model’s experts and $\text{top-}k$, the published speed, the bound at datasheet-physical ceilings (GPU datasheet bandwidth, the top of the host-DRAM band, datasheet PCIe, no latencies; `scripts/audit.py`), how the optimum’s reads were obtained (*trace*: Belady’s MIN with bypass on an on-policy routing trace of the model; *uniform*: independent uniform routing, which overstates the reads and so the percentage; *no slots*: no GPU expert capacity, the CPU-only rate is the bound; *all fit*: every expert resident, nothing offloaded) and the percentage of the bound. Medians: trace 9.5% ($n=29$), uniform 34.8% (11), no slots 49.0% (8), all fit 4.0% (4), all rows 14.5% (52). Rows below 5% are more likely configuration mismatches with the bound’s assumptions (precision, dense layers on the CPU, end-to-end timing) than inefficiency, and are not adjudicated.

System	Model	GPU	C	E/k	tok/s	bound	reads	%
Fiddler [19]	Mixtral-8x7B	Quadro RTX 6000	1	8/2	2.5	17	trace	14.6
	Mixtral-8x7B	RTX 6000 Ada	3	8/2	8.3	57	trace	14.4
ProMoE [35]	DeepSeek-V2-Lite	RTX 4090	45	64/6	49.9	216	trace	23.1
	Qwen1.5-MoE-A2.7B	RTX 4090	46	60/4	67.2	216	trace	31.1
	Qwen2-57B-A14B	RTX 4090	42	64/8	26.9	136	trace	19.8
	Mixtral-8x7B	RTX 4090	6	8/2	27.0	152	trace	17.8
KTransformers [5]	DeepSeek-V3-0324	A100 40GB	0	256/8	5.9	12	no slots	48.8
	DeepSeek-V2.5-1210	A100 40GB	0	160/6	11.9	24	no slots	49.2
	Qwen2-57B-A14B	A100 40GB	0	64/8	22.9	35	no slots	64.9
	DeepSeek-V3-0324	RTX 4080	0	256/8	16.1	33	no slots	48.3
	DeepSeek-V2.5-1210	RTX 4080	0	160/6	17.6	32	no slots	55.3
	Qwen2-57B-A14B	RTX 4080	0	64/8	37.0	48	no slots	76.9
HybriMoE [53]	DeepSeek-V2-Lite-Chat	RTX A6000	16	64/6	14.5	470	trace	3.1
	DeepSeek-V2-Lite-Chat	RTX A6000	48	64/6	15.3	655	trace	2.3
	Mixtral-8x7B-Instruct	RTX A6000	2	8/2	4.4	53	trace	8.2
	Mixtral-8x7B-Instruct	RTX A6000	6	8/2	7.3	133	trace	5.5
	Qwen2-57B-A14B-Instruct	RTX A6000	16	64/8	9.6	86	trace	11.1
	Qwen2-57B-A14B-Instruct	RTX A6000	48	64/8	9.0	116	trace	7.7
Fate [9]	Qwen1.5-MoE-A2.7B	RTX 3090	46	60/4	14.0	198	trace	7.1
	Qwen1.5-MoE-A2.7B	GTX 1080 Ti	15	60/4	3.7	76	trace	4.9
SP-MoE [6]	Mixtral-8x7B	RTX 3090	1	8/2	1.2	16	trace	7.7
	Mixtral-8x7B	RTX 4090	1	8/2	1.3	13	trace	9.5
	Mixtral-8x7B	A100 40GB	3	8/2	1.5	32	trace	4.5

continued

System	Model	GPU	C	E/k	tok/s	bound	reads	%
MoE-SpeQ [39]	Phi-3.5-MoE	RTX 4090	3	16/2	3.3	57	trace	5.8
	DeepSeek-V2-Lite	A100 40GB	7	64/6	5.6	142	trace	3.9
	DeepSeek-V2-Lite	A100 40GB	64	64/6	10.1	335	all fit	3.0
	DeepSeek-V2-Lite	A100 40GB	64	64/6	13.8	335	all fit	4.1
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	13.2	335	all fit	3.9
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	28.3	335	all fit	8.5
CPU-GPU collab. [15]	Phi-3.5-MoE	A100 40GB	7	16/2	6.0	129	trace	4.7
	Phi-3.5-MoE	A100 40GB	7	16/2	6.1	129	trace	4.7
	Mixtral-8x7B	RTX 4090	1	8/2	4.8	13	trace	36.5
FreeToken [46]	Phi-3.5-MoE	RTX 4090	3	16/2	10.4	56	trace	18.6
	Qwen3.6-35B-A3B	RTX 5090	94	256/8	77.1	227	uniform	34.0
	DeepSeek-V4-Flash	RTX 5090	28	256/6	24.9	81	uniform	30.8
	Qwen3.6-35B-A3B	RTX 5090	104	256/8	73.8	205	uniform	36.1
	Qwen3.6-35B-A3B	RTX 4090	72	256/8	42.9	123	uniform	34.8
	Qwen3.6-35B-A3B	RTX 4060 Laptop	60	256/8	39.3	97	uniform	40.4
SeqMoE [41]	GLM-5.2	RTX PRO 6000 Blackwell	36	256/8	14.9	30	uniform	49.9
	Qwen3-30B-A3B-FP8	RTX 4090	57	128/8	104.1	374	trace	27.9
	Qwen3-30B-A3B-FP8	RTX 4090	19	128/8	47.9	374	trace	12.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	102	256/8	118.3	305	uniform	38.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	38	256/8	70.2	212	uniform	33.2
	GPT-OSS-120B-MXFP4	RTX PRO 6000 Blackwell	57	128/4	139.2	378	trace	36.8
Pipelined sharding [38]	DeepSeek-V4-Flash	RTX PRO 6000 Blackwell	115	256/6	57.4	183	uniform	31.3
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	44	128/8	32.1	1032	trace	3.1
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	0	128/8	7.7	40	no slots	19.3
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	42	128/8	11.5	108	uniform	10.7
	Qwen3-30B-A3B-Instruct-2507	RTX 5070 Ti	107	128/8	54.9	511	trace	10.8
llama.cpp fork (leloch) [21]	Laguna S 2.1	RTX 5090	66	256/10	31.3	88	uniform	35.5
llama.cpp PR 27861 [7]	Qwen3-30B-A3B Q4_K_M derivative	Radeon RX 7600 8GB	32	128/8	16.5	169	trace	9.8

H Artifact description

- **System.** A patch to llama.cpp commit 4da6337 (the expert cache, FETCH, the mailbox and the llama-ec-bench instrumentation: teacher-forced loss, logit dumps, routing lookahead records); built with CUDA 13.0 for compute capability 12.0.
- **Jobs.** One shell script per job; each installs FreeToken at commit 0d652e7, downloads the models from Hugging Face, runs the bandwidth probes, writes any on-machine prediction, runs the client, and stores raw rows (one JSON line per problem), server logs and engine counters. A small runner rents a Vast instance, streams a heartbeat, fetches the results and only then destroys the instance.
- **Analysis.** scripts/*_stats.py compute every interval from the raw rows; scripts/perlayer_model.py, speed_limit_v2.py, shapley_gap.py, policy_study.py, foresight_exact.py, batchk_trace.py, fig_audit_sol.py and scorecard.py produce the model comparison, the limit, the accounting, the trace studies, the audit figure and the scorecard; scripts/wsg_tables.py and wsg_numbers2.py generate every table, figure and number in this paper from their outputs. The audit’s 52 published rows and the physical speed-of-light model they are scored against are data/published_measurements.json, data/audit/ and scripts/audit.py.
- **Cost.** The 57 rentals (35 distinct machines) of jobs 058–095 cost 46.3 US dollars at their hourly GPU prices, by the ledger in the repository; storage and download charges, which the ledger omits, add to this.